

claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\workflows\wf_f9aa1d1c-c83\agent-  
a8215065c2526f7ee.jsonl`  
- SHA-256: `c5356e198f64ce0912b3fa69d0da574c41663bcc354f6c4da161ba9d83dc670e`  
- Source modified: `2026-06-09T11:28:15+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_f9aa1d1c-c83`  
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`
```

Transcript

1. user (2026-06-09T11:25:35.357Z)

DIMENSION = IMPLEMENTATION LANGUAGE for the upcoming work, for a SOLO founder whose stack is Python/FastAPI. Use WebSearch+WebFetch where useful. Three layers: (1) ML TRAINING+EVAL (PyTorch/ms-swift ? is this Python-locked?); (2) PLATFORM (async job orchestration, multi-tenant API, storage/compute backends ? Python/FastAPI vs Go/Rust for hot paths); (3) ON-DEVICE inference LATER (exported model via ONNX Runtime / llama.cpp / TFLite / Core ML ? does this dictate the dev language?). Recommend: stay Python everywhere, or introduce another language for specific layers ? and WHEN that's justified vs premature. Be pragmatic about solo-founder velocity.

OWNER CONTEXT:

OWNER'S REFRAMED VISION (from PR #61 review comments ? supersedes earlier framing where they differ):

- IMMEDIATE GOAL: a TRAINING FRAMEWORK + HARNESS to produce a VERY HIGH precision/recall model that extracts

- HIGHLIGHTS and RALLIES from a clip. Quality first. On-device is LATER (a model-compression goal for power users to

- save upload cost) ? NOT the immediate goal; don't over-index on it now.

- MULTISPORT FROM THE START: all NET-SEPARATED RACQUET SPORTS (badminton, tennis, table tennis, pickleball, padel)

- first, eventually ALL sports. A SINGLE, sports-AGNOSTIC model (the collector + indexer are already sport-generic by

- design; the repo will be renamed off "badminton"). More labels/compute/storage is an accepted "good problem."

- CONVERSATIONAL VIDEO-QA is a north-star feature: upload a video -> the model processes it -> the user asks CONTEXTUAL

- QUESTIONS continuously, e.g. "show me the longest rally", "make a video of all the service faults in this clip",

- "show me a clip where the rally lasted over 20 shots". This implies per-video

- STATE/bookkeeping the model itself

- won't hold ? so the FRAMEWORK must be designed for that bookkeeping from the start. Owner asks: feasible to embark on

- now, or punt and revisit after quality is solid?

- HARD COMMERCIAL-LICENSE GUARDRAIL (owner: "never corrupted"): EVERY dependency ? code, data, AND model weights ? must

- be usable in PRIVATE PROPRIETARY COMMERCIAL software. MIT / Apache-2.0 / BSD are the easy fits. Reject viral/NC/

- custom-restrictive (e.g. Gemma-3 viral terms; weights trained on paid-LLM outputs; >X-MAU caps if truly binding).

- NEVER train on paid-LLM (Gemini/OpenAI/Anthropic) outputs. Exclude minors; separate training opt-in. Hopefully a

- CONVERSATIONAL single model.

- CURRENT STATE (de-risking evidence): on a 6-match human-labeled corpus, a learned per-frame prototype gets held-out

- AUC 0.83-0.92 and a cross-video LOVO mean 0.583 vs the heuristic's 0.480 ? and the learned

model DECOUPLES from the

multi-court "background games" confound the heuristic can't escape. WASB = the shuttle detector (MIT code, but the

distributed checkpoint is academic-data-trained ? provenance unresolved = a commercial-cleanliness blocker to fix).

- CONSTRAINTS: solo founder, Bangalore; local dev box = ONE RTX 2070 Super (8GB) + WSL2; paid Gemini stays the fallback;

cloud GPU rented per-job (Modal/RunPod per the earlier study).

REQUIREMENTS BRIEF:

```
{"requirements":["GOAL (reframed): The immediate deliverable is a TRAINING FRAMEWORK + HARNESS that produces a VERY high precision/recall model for extracting HIGHLIGHTS and RALLIES from a clip. Quality is the first-order objective; everything else is secondary to model quality.", "ON-DEVICE IS DEFERRED (reframe vs earlier 'on-device = immediate moat'): On-device / model-compression is a LATER goal, motivated as an upload-cost saver for power users. It is NOT the immediate target and the strategy must NOT over-index on it now. (Supersedes OWNED_MODEL_IMPLEMENTATION_PLAN.md's 'on-device = the privacy moat / immediate' framing.)", "MULTISPORT FROM DAY ONE: Build for ALL net-separated racquet sports first (badminton, tennis, table tennis, pickleball, padel), with a path to eventually ALL sports. No single-sport (badminton-only) scoping.", "SINGLE SPORT-AGNOSTIC MODEL: Target ONE sports-agnostic model rather than per-sport models. Collector + indexer are already sport-generic by design; the repo will be RENAMED off 'badminton'. Generic data schema stays; coach->athlete remains an optional overlay, not baked in.", "SCALE COST ACCEPTED: More labels / compute / storage from multisport breadth is an explicitly accepted 'good problem' ? do not constrain the design to minimize label/compute/storage volume at the expense of breadth or quality.", "CONVERSATIONAL VIDEO-QA IS THE NORTH-STAR FEATURE: upload a video -> model processes it -> user asks CONTEXTUAL questions CONTINUOUSLY (e.g. 'show me the longest rally', 'make a video of all the service faults', 'show me a clip where the rally lasted over 20 shots'). Hopefully served by a single conversational model.", "PER-VIDEO STATE/BOOKKEEPING IS A FRAMEWORK REQUIREMENT: The conversational-QA loop implies durable per-video state (rally inventory, shot counts, event/fault tags, timestamps, derived clips) that the MODEL ITSELF will not hold. The FRAMEWORK must be DESIGNED FOR THIS BOOKKEEPING FROM THE START, even if the QA feature ships later.", "OPEN DECISION the owner is asking for a recommendation on: is conversational-QA feasible to EMBARK ON NOW, or should it be PUNTED and revisited AFTER rally/highlight quality is solid? (The bookkeeping-from-the-start requirement holds either way.)", "HARD COMMERCIAL-LICENSE GUARDRAIL ('never corrupted'): EVERY dependency ? code, training data, AND model weights ? must be usable in PRIVATE, PROPRIETARY, COMMERCIAL software. MIT / Apache-2.0 / BSD are the accepted easy fits.", "REJECT viral / non-commercial / custom-restrictive licenses: explicitly reject Gemma-3 viral terms, weights trained on paid-LLM outputs, and >X-MAU caps IF truly binding for commercial use.", "NEVER TRAIN ON PAID-LLM OUTPUTS: Gemini / OpenAI / Anthropic outputs must never be a training-data source (terms/copyright). Paid Gemini remains inference/serving/FALLBACK only. (Also: the existing Gemini-silver training labels in training.label_candidates must be purged before they bake in ? M0.3.)", "DATA-ELIGIBILITY GUARDRAILS: Exclude minors from the training pool; per-user training data requires a SEPARATE explicit opt-in. Split data by match.", "QUALITY EVIDENCE / STARTING POINT (de-risking baseline to beat): On a 6-match human-labeled corpus, the learned per-frame prototype achieves held-out AUC 0.83-0.92 and cross-video LOVO mean 0.583 vs the heuristic's 0.480. The learned model is the chosen path BECAUSE it DECOUPLES from the multi-court 'background games' confound that the heuristic cannot escape (3 heuristic levers ? spatial gate, audio-foreground, court-region separability ? were each tested and failed this session; the problem is temporal, not spatial).", "WASB PROVENANCE IS AN OPEN COMMERCIAL-CLEANLINESS BLOCKER: WASB is the shuttle/ball detector (MIT code), but the distributed checkpoint is academic-data-trained with unresolved provenance (the C3 item) ? this is a commercial-cleanliness blocker that must be resolved (document as Gen-0 prerequisite + plan own-weights, or scope an alternative.)", "RESOURCE/OPERATING CONSTRAINTS: Solo founder, Bangalore. Local dev box = ONE RTX 2070 Super (8GB VRAM) + WSL2 (no full-pipeline run in the dev container ? no GPU/torch/cv2/numpy there). Cloud GPU rented per-job (Modal / RunPod per the earlier study). Paid Gemini stays the fallback.", "LANGUAGE/SCOPE NOTE: The reframed vision adds no new natural-language/multilingual requirement beyond English conversational-QA phrasing; 'language' as a strategic axis is limited to the conversational-QA query interface, not localization.", "tensions":["QUALITY-FIRST vs ON-DEVICE-LATER vs HARDWARE: The very-high-quality goal pushes toward larger VLM bases and cloud training, but the only local box is an 8GB RTX 2070 Super and on-device is explicitly deferred ? so the immediate model is NOT constrained to fit 8GB, yet the eventual compression goal must remain reachable. The strategy must pick a base that is both high-quality-trainable AND later-compressible without re-architecting.", "MULTISPORT-FROM-START vs CURRENT EVIDENCE DEPTH: The de-
```

risking corpus is badminton-only (WASB shuttle features, multi-court badminton confound). A single sport-agnostic model is required from day one, but the only validated signal/features and the only labeled corpus are badminton ? the feature stack (WASB trajectory) may not generalize to tennis/table-tennis/padel, and per-sport corpora don't exist yet.", "SINGLE SPORT-AGNOSTIC MODEL vs COMMERCIAL-CLEAN DETECTORS: A sport-agnostic model likely needs per-sport object/ball detectors or a unified detector; WASB (badminton) already has unresolved provenance (C3), and clean MIT/Apache/BSD detectors for tennis/TT/pickleball/padel are not yet identified ? each new sport reopens the license-cleanliness guardrail.", "CONVERSATIONAL-QA NORTH-STAR vs 'NEVER TRAIN ON PAID-LLM': A single conversational model that answers contextual video questions typically wants an LLM/VLM teacher, but the strongest available teachers (Gemini/OpenAI/Anthropic) are forbidden as a training source ? so the conversational capability must come from an Apache-2.0/MIT base + own data, narrowing the candidate set and raising the quality bar to clear without paid-LLM distillation.", "BOOKKEEPING-FROM-THE-START vs SEQUENCING DISCIPLINE: The owner wants the framework designed for per-video QA state NOW, but the committed next slice is the async job model (single-tenant) and heavy model work is gated on scaffolding + sign-off ? designing durable QA-state bookkeeping prematurely risks building against an architecture (and a model) that doesn't exist yet (the 'embark now vs punt' question the owner is explicitly asking).", "ACCEPTED SCALE COST vs SOLO-FOUNDER / SINGLE-GPU BUDGET: 'More labels/compute/storage is a good problem' collides with a solo founder on one 8GB GPU renting cloud per-job ? multisport label volume and storage are accepted in principle but still bounded by one person's labeling throughput and per-job cloud spend.", "SINGLE CONVERSATIONAL MODEL' (hoped) vs HARNESS-FIRST QUALITY GOAL: The owner hopes for one model that does detection + segmentation + conversational QA, but the harness/quality goal is concretely about a rally/highlight extractor; folding conversational QA into the same model vs keeping extraction (model) and QA (bookkeeping + retrieval/LLM layer) separate is an unresolved architectural fork.", "QUALITY-FIRST ('very high precision/recall') vs HONEST EVIDENCE CEILING: Current best cross-video LOVO is ~0.583 (directional, $n \leq 6$, linear, threshold-transfer unsolved) ? a large gap from 'very high precision/recall', so the quality target may require substantially more data/features/base-model than the current de-risking artifacts demonstrate."]]

2. assistant (2026-06-09T11:25:38.900Z)

I'll research this implementation-language decision. Let me start by reading the relevant project docs to ground my analysis in the actual codebase state, then do targeted web research on the parts where current facts matter (ms-swift Python-locking, on-device runtime constraints).

3. user (2026-06-09T11:25:39.981Z)

docs\EVALUATION.md
docs\TRACKNET_WSL_SETUP.md
docs\GOLDEN_SET_IMPLEMENTATION.md
docs\GOLDEN_SET_PHASE1_VIDEOS.md
docs\HOW_RALLY_DETECTION_WORKS.md
docs\archives\COMMERCIAL_READINESS_REVIEW.md
docs\archives\MONETIZATION_AUDIT.md
docs\archives\README.md
docs\DEFERRED_HARDENING_PLAN.md
docs\PMF_MARKET_RESEARCH.md
docs\ROADMAP.md
docs\archives\checkpoints\2026-06-eval-baseline\README.md
docs\archives\checkpoints\2026-06-logging-reporting\README.md
docs\archives\checkpoints\2026-06-scaffolding-complete\README.md
docs\archives\decisions\DECISIONS.md
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md
docs\archives\past_projects\README.md
docs\archives\research\AUDIO_E1_FINDINGS.md
docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md
docs\archives\research\RALLY_DETECTION_RESEARCH.md
docs\COMMERCIALIZATION.md
docs\PLATFORM_ARCHITECTURE.md
docs\BACKLOG.md
docs\GOLDEN_SET_VENDURING.md
docs\README.md
docs\VIDEO_RECORDING_GUIDELINES.md

docs\archives\checkpoints\2026-06-strategy-foundation\README.md
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md
docs\archives\quality-iterations\2026-06-initial\README.md
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md
docs\CODE_MAP.md
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md
docs\archives\quality-iterations\README.md

4. user (2026-06-09T11:25:40.381Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-255 of 330 total (27423 tokens, cap 25000). Call Read with offset=256 limit=255 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```

1      # CODE_MAP.md
2
3      **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-
hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
4
5      **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this
doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure
(this revision follows #40).
6
7      **LEGEND**
8      - `@path` = file (always repo-relative)
9      - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
10     - `->` = dataflow / call / "produces"
11     - `$` = data contract (canonical shape)
12     - `?` = gotcha / footgun
13     - `?` = registry / plugin extension point
14     - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
15
16     ---
17
18     ## 1. PIPELINE
19
20     ### 1A. Runtime: video file -> highlight reel
21     ```
22     typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
23                                     @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
24     -> video_id = MD5(whole file)          @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
25     -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
26     FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
27     [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
28     -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
29     -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')
30     -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
31     -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
32     [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
33     -> finally: emit_indexer_report(...) @backend/reporting/__init__.py (? ALWAYS runs

```

```

? even on validation failure / exception; NEVER raises; grafts response["reports"])
34     -> select segment ids -> [(start,end)] (+ stitching padding)
35     -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
36     per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
37     ```
38     Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
39     ```
40     P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
41     -> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted
via db.add_candidate_segment]
42     -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt) ? provider_registry
43     -> P4 db.add_play_segment + db.link_candidate_to_segment
44     ```
45     Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
46     Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).
47
48     ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
49     ```
50     WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
51     -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
52     -> WasbRunner.run_predict(video, out_dir)
53     stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...`
54     -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
55     -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
56     -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING
BRAIN]
57     ```
58     (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
59
60     ### 1C. Calibration / eval flow (NO pipeline run unless told)
61     ```
62     GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
63     DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval] stage
? {candidates, final}
64     -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
65     -> harness.evaluate -> EvalReport -> report_to_dict
66     -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
67     -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py
[CI gate]
68     Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
69     Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned
classifier) @backend/eval/{calibrate_local,distill_local}.py
70     Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
71     Gemini refinement driver (tuning, standalone):
gemini_refine.{refine_window,full_video_rallies} @backend/eval/gemini_refine.py
72     Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
73     ```
74     Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch,

```

```

can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
75
76     ---
77
78     ## 2. SUBSYSTEMS
79
80     ### 2.0 Orchestration & config
81     - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS).
Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig`
(INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces
stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI
summaries may still use `print` for direct output.
82     - `::app` FastAPI; `::db_instance`, `::global_config` (`Optional[AppConfig]`) module
globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` ->
every endpoint NPES.
83     - `::process_video` `@POST /api/process` -> validate -> add_video -> segment ->
`finally:` always `emit_indexer_report` (never raises). `::compile_highlights` `@POST
/api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`.
`::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs
`indexing.motion_threshold` (dual model/dict read).
84     - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
85     - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
86     - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
87     - `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()`
(blocks automation), skips validation. Config via `backend.config.load_config` (typed);
segmenter, padding, and db path all read from the model (no literal fallbacks).
88     - `@run_eval.py::main` ? score one video's DB preds vs GT CSV.
`--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed.
`::default_db_path` resolves via `backend.config.load_config`
(`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
89     - `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate.
`load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label);
config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig`
(same object the live pipeline feeds segmenters).
90     - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic:
**0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare
(snapshots even a regressing run); `::default_db_path` resolves via
`backend.config.load_config`.
91
92     - `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports
`load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`,
`ValidationConfig` (from `.models`); `__all__` = exactly those 6. Use `from backend.config
import load_config, AppConfig`.
93     - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for
defaults**.
94     - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`,
`ai_model='gemini-2.5-flash'`; sections `validation`, `indexing`, `output`, `stitching`
(`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ?
`extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
95     - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain
dicts** (`model_dump(mode='json')`, NOT the model) so chained
`config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any
section (sections discovered dynamically from `model_fields`). ? "bit every validator on real
runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises
`KeyError` for unknown top-level keys else delegates to `.get`.

```

```

96     - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`,
`min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`,
`chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`,
`ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested
`tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `gemini`. ?
`::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ?
a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`
97     - `::ValidationConfig` (`min_resolution_width=1280`, `min_resolution_height=720`, `min_fps=29.9`, `min_blur_threshold=20.0`, `max_scene_cuts_per_minute=0.5`, nested `relevance`);
`::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`).
`::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`,
`conda_env='TrackNetV4'`, `weights_path='', `queue_length=5`, `velocity_threshold=0.02`).
`::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`,
`output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`).
`::StitchingConfig`
(`begin_padding=0.5`, `end_padding=0.5`, `use_cache=False`, `min_shot_count=1`). `::Sport` Literal
of 5 sports. `::Config = AppConfig` alias.
98     - `@backend/config/loader.py` ? load/save with defaults-merge.
99     - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT repo-root).
`::get_built_in_defaults` = `AppConfig().model_dump(...)`
100    - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-
level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole
defaults dict for that section (Pydantic then re-applies field defaults for missing leaves).
Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-
fatal).
101    - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present
(used by setup.py/--setup). `::get_default_config` ? ? transitional plain-dict alias.
102    - `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**`indexing.default_segmenter='gemini'`** diverges from model default `yolo_hybrid` ? file
wins at runtime.
103    - `@setup.py` ? `::init_default_config` ? delegates to
`backend.config.save_default_config` (single-source defaults; skips if config.json exists).
`::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8,
ffmpeg/ffprobe on PATH, optional torch+CUDA).
104
105    ### 2.1 Segmenters (under pipeline/segmenters) ?
106    - `@backend/pipeline/segmenters/base.py`
107    - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`, `description`, `process_video`. $ `process_video(video_id, video_path, player_pool=None,
max_frames=None)` -> List[dict{id, start_time, end_time, duration, metadata}]`.
108    - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new
segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module
bottom, AND import the module in `@backend/pipeline/segmenters/_init_.py` (registration is an
IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `name` ->
name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-
instantiates + swallows exceptions into "No description available." (a throwing `description` is
masked).
109    - `@backend/pipeline/segmenters/_init_.py` ? imports all 5
(`motion, gemini, yolo_hybrid, tracknet_hybrid, wasb_hybrid`) = the de-facto registration manifest;
re-exports them + `segmenter_registry` via `__all__`.
110    - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`wasb_hybrid`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE
default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config
`indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs
`{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike
tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit
(copy-paste drift).
111    - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`tracknet_hybrid`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
112    - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name

```

```

'yolo_hybrid', no YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005).
`::PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections).
`::lazy_load_model` (lazy `torchvision` import so module/tests load without it),
`::extract_action_windows_from_chunk`, `::make_tracker` (per-chunk ByteTrack ? track IDs
chunk-local), `::DETECTOR_FACTORIES`/`::DEFAULT_DETECTOR`='fasterrcnn_resnet50_fpn'. ?
velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to
`frame_skip`; pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU);
`yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated
aliases still honored.
113 - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `gemini`) ?
pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
114 - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `motion`) ?
pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
115
116 ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
117 - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() ->
Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately
matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring
includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on
threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL
de-risk seam (low-regret 6 / review item 1).
118 - `@backend/pipeline/detectors/tracknet_runner.py`
119 - `::TrackNetConfig`(+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro`
field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
120 - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/`~` passthrough, UNC unhandled),
`::TrajectoryPoint`(frame,visible,x,y).
121 - **`::parse_trajectory_csv` @staticmethod** + **`::trajectory_to_action_windows`
@staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path`
defaults `` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names
`<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`,
`frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout
(hangs forever). `::stage_video` "OK" sentinel.
122 - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner)
(+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default
`wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::INFER_WIN`) +
video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` =
`staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-
named `<stem>.wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until
staged); `wsl_bash` duplicated (drift risk). Both runners now satisfy the DetectorRunner ABC
for future swappability.
123 - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT
repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache:
`::DET_FILE`='det_raw.jsonl', `::MANIFEST_FILE`='manifest.jsonl', `::CACHE_VERSION`=1,
`::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to
heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights`
compared raw), `::atomic_write_text`/`_atomic_write_trajectory`, `::dets_to_tracker` (xy MUST be
np.array), `::build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW),
`::default_cache_dir` (`<stem>.wasbcache` next to out). ? GPU pass resumable; CPU tracker
STATEFUL -> always fully re-run; `--fresh` ignores cache.
124 - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure,
no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> `x`),
`::count_net_crossings` (sign changes vs median net; deadband jitter zone),
`::gate_windows`/`::apply_gate` (`min_crossings=2` default; kills single-toss service feeds).
`::GatedWindow`(start,end,crossings,visible_points,kept). ? window contract here is
(start,end) tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local /
distill_local / export_wasb_segments.
125
126 ### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
127 - `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`,
`::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict`) = wire shape),

```

```

`::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN)
for mean_iou/boundary errors.
128 - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end`
CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
129 - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video
k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video;
needs 22 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT),
`::select_best`, `::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. $
`PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap
>=0.1` = overfit alarm.
130 - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfigs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` ($`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
131 - `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing +
net-crossing gate, no cloud at runtime). `::local_grid` (180 cfigs), `::make_local_predict_fn`
(adds `rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a
labels file yields no rallies), `::run`/`::main`. $
`LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.
132 - `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison
+ optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video),
`::SEG_FIELDS`, `::COMP_FIELDS`, `::build_comparison`, `::seconds_to_mmss`, `::maybe_slice`
(lazy-imports rally_slicer). `--slice` requires `--video`. ? `write_segments_csv` output loads
straight into `rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is
load-bearing.
133 - `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in
run_phase3_eval). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1),
`::default_stages_for` (`auto`; `::FINAL_ONLY_SEGMENTERS={motion,gemini}`),
`::resolve_video_path` (normalizes collector `./data`.. Windows paths), `::format_aggregate`.
134 - `@backend/eval/harness.py` ? DB-pred scorer (no re-run).
`::STAGES=('candidates','final')`, `::get_predictions`
(candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})`); ? unknown
stage raises ValueError; reused by regression.py),
`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate), `::format_report_text`.
135 - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES`
(5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default
0.0), `::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->(X,y,intervals); `feature_fn` is the substrate plug-in seam.
136 - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `_sigmoid`
clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON "model":"logreg" tag.
137 - `@backend/eval/eval_partial_labels.py` ? score pipeline vs **PARTIAL human labels**
(PR #60). `::restrict_to_window` (drop preds outside `[0, last GT end]` so un-labeled-tail
detections aren't FPs ? THE partial-label methodology), `::run` (baseline/+gate/tuned/+gate ops
table + per-rally diff via `export_wasb_segments.build_comparison`), `::sweep`/`--sweep` (grid
vs labels; ? fit-on-self = optimistic), `::DEFAULT_GRID`, `::TUNED=(0.05,1.0,1.0)`. ?
`window_end` inferred from `max(GT end)` is WRONG if a video is labelled PAST its last rally ?
P1 fix = collector persists `labeled_window` in the manifest. Reuses calibrate_wasb + rally_gate
+ metrics.
138 - `@backend/eval/rally_seq_proto.py` ? **PROTOTYPE** learned per-FRAME rally segmenter =
owned-model **Gen-0 seed** (PR #61). 6 features over the WASB trajectory (`vis_rate, spd_mean,
spd_max, cross_rate, x_std, y_std`) -> `classifier.LogisticRegression`, honest LOVO.
`::build_frame_arrays`, `::features` (scipy `uniform/maximum_filter1d` windowed; ? speed NOT
fps-normalized yet ? 30 vs 60fps), `::labels_for`, `::probs_to_segments`
(smooth->threshold->merge/min-dur), `::roc_auc` (rank-based, no sklearn), `::run` (per-video AUC
+ LOVO-threshold F1 + oracle-threshold F1). ? DIRECTIONAL only (n?5, linear, threshold-transfer
unsolved); held-out per-frame AUC 0.83?0.92, **beats the heuristic on multi-court footage**
(Kushagra/gBaaddy). Manifest `output/human_lovo_manifest.json` (the n=5 human corpus) shared w/
calibrate_local.

```

```

139 - ? **MULTI-COURT / background-game confound (2026-06-08 finding):** WASB tracks EVERY
shuttle in frame incl. other courts; videos with background games (Kushagra, gBaaddy) have high
dead-time shuttle visibility (41?57% vs 18?22% clean) ? the velocity-windowing heuristic can't
find clean rally boundaries (F1 0.16/0.28 vs ~0.75 on single-court). Diagnostic = in-rally÷dead-
time visibility "contrast" (?3.6× heuristic works, ?1.9× fails). Lever = **foreground-court
isolation** (spatial gate to the prominent court, needs real court-region detection). Academy =
multi-court = the target env. See `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
140 - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond
calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT:
duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal),
`::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`),
`::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-
one-video-out needs ?2 videos; final model trained on all.
141 - `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT
padded clips (precision lever; eval/tuning, NOT the live pipeline).
`::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (?
FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`,
`::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises
SystemExit); imports `BASELINE` from calibrate_wasb +
`TUNED`/`write_segments_csv`/`seconds_to_mmss` from export_wasb_segments.
142 - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`,
`::regress_with_provider` (GT via ?`annotation_registry.get(tier)`;
"file"->FileProvider/"vendor"->HumanVendor/"auto"->oracle),
`::save_baseline`/`::load_baselines`, `::RegressionResult`. ? imports `annotation_registry` from
**`backend.annotations`** NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass).
143
144 ### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
145 - `@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector. Pure
numpy + stdlib `wave` (NO scipy); ffmpeg only for extraction. `::HitEvent`(t,strength),
`::extract_audio` (ffmpeg -> mono 16k PCM WAV), `::load_wav`, `::onset_envelope` (pure-numpy
STFT spectral flux; `band=(lo,hi)` band-limit), `::detect_hits` (adaptive `mean+k.std`;
`k`=sensitivity knob), `::detect_hits_in_video`. ? separate signal ? WASB never modified by
this; per audio-e1-findings recall 100% but discrimination ~2.2×, band-pass null -> NOT adopted
(PR #35).
146 - `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
147
148 ### 2.4 Stitcher / tools / utils
149 - `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::parse_time`
(mirrors gemini.parse_time; ? unparseable -> 0.0), `::get_video_duration`. ? if duration probe
fails (0.0) NO boundary validation; `begin_padding`(0.5)/`end_padding`(1.0) defaults; temp clips
in a TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses
`print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so
final concat can stream-copy ? change a preset and concat may break.
150 - `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure,
unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::ClipWindow`(rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (?
duplicate compiler's by copy). `_read_time` -> backend/eval/annotations.parse_timestamp.
151 - `@backend/utils/video.py` ? `::get_video_duration` (ffprobe; ? 0.0 on failure =
"unknown"), `::extract_video_chunk(...,reencode=False,scale_width=None)`. ? copy mode cuts at
nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale_width` requires
`reencode=True`.
152 - `@backend/utils/geometry.py` ? `::get_velocity_normalised` (fraction of frame-width/s;
? raw-px fallback if width<=0; used by yolo_hybrid), `::calculate_iou` (? +1 px convention
inflates IoU), `::get_center/get_distance/get_velocity`.
153
154 ### 2.5 Validators ?
155 - `@backend/pipeline/validators/base.py::VideoValidator` ABC, `::ValidationResult`
(pydantic: validator_name,passed,message,details), `::ValidationRegistry` (ordered LIST ?
registration order = exec order; ? no dedup), `::registry` (singleton). § `validate(video_path,
```

```

config, context) -> ValidationResult`. `::run_all_with_context -> (results, context)` (wraps
each validate in try/except -> failed result not abort); `::run_all` discards context. ? **add a
validator**: subclass + `registry.register(MyValidator())`.
156 - `@backend/pipeline/validators/__init__.py` ? registers in EXEC order: FormatValidator,
ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator,
RelevanceValidator (? producers before consumers ? format_check first; ? import mutates global
registry as side effect).
157 - `@backend/pipeline/validators/format.py::FormatValidator` (`format_check`) ?
**PRODUCER** of `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is
substring `mp4` (so MOV-family passes); external `ffprobe`.
158 - `@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator`
(`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails "Run format_check first" if
absent). `min_fps` 29.9; min res 1280x720.
159 - `@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator`
(`pts_continuity_check`) ? re-probes (no context). ? 10.0s keyframe-gap threshold hardcoded
(comment says 8.0s ? drift, NOT config-driven); any backward jump fails; `<2` keyframes PASSES
(too short).
160 - `@backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator`
(`scene_cut_check`) ? OpenCV. ? `cut_threshold=0.6` hardcoded (NOT config); samples ~2fps, caps
at 100 SAMPLED frames (~first 50s); `max_scene_cuts_per_minute` (0.5) is the config fail
threshold.
161 - `@backend/pipeline/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var <
`min_blur_threshold` (default 100.0 in-file, 20.0 in models/config) = fail; samples 15 frames. ?
magic absolute, not resolution-normalized.
162 - `@backend/pipeline/validators/relevance.py::RelevanceValidator` (`relevance_check`) ?
HSV court-green ratio (? purely color, no geometry despite name); lazy `import backend.sports`;
3-tier config precedence (`validation.relevance` -> sport profile -> hardcoded); ? unknown sport
-> empty cfg, falls back to defaults, does NOT fail.
163
164 ### 2.6 Sports / Providers / Annotations ?
165 - `@backend/sports/base.py::SportProfile` ABC
(`name`, `get_prompt`, `get_relevance_config`);
`@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
`@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton; ?
`register` instantiates profile to read `name`). ? **add a sport**: subclass `SportProfile`,
`sport_registry.register(MyProfile)`. $ prompt emits JSON array `{start_time(float DECIMAL SEC),
end_time, shuttle_exchange_count(int), shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt
actively defends against MM:SS; `get()` returns a fresh instance each call.
166 - `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`);
`@backend/providers/gemini.py::GeminiProvider` (lazy `::client` property; `::MAX_UPLOAD_MB=500`;
upload->poll->generate JSON temp 0->delete); `@backend/providers/__init__.py::provider_registry`
(+`::ProviderRegistry`, auto-registers `gemini`). ? **add a provider**: subclass,
`provider_registry.register('name', MyCls)` (key passed EXPLICITLY); `get(name, api_key,
model_name)`. $ `analyze_video(path, prompt, chunk_duration=0.0, label='')` -> (segments:list,
metrics:dict{upload_time,process_time,gen_time}). ? blocking 10s poll; returns `([], metrics)`
on ANY failure (empty != error); oversize clip silently skipped; orphaned remote file possible
if process dies mid-run.
167 - `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`),
`::Interval` (= metrics.Interval), `::QUEUED/RUNNING/DONE/FAILED`/`::STATUSES`,
`::AnnotationProviderRegistry` (decorator-friendly `register` returns the class; ? keys off
`provider_cls().name`, falls back to `__name__` if no-arg ctor throws; `list_available()` ->
LIST not dict), `::annotation_registry`. ? **add a tier**: subclass (define `name`),
`annotation_registry.register(MyCls)` at module bottom + import in `__init__.py`. $ `fetch` ->
List[Interval] sorted; contract says RAISE (not `[]`) on unavailable.
168 - `@backend/annotations/file_provider.py::FileProvider` (`file`; job_id IS the
resolved CSV path; `resolve` -> `<csv_dir>/<video_ref>.csv`; ->
`backend.eval.annotations.load_annotations`; ? `guideline`/`sport` ignored).
169 - `@backend/annotations/automated_provider.py::AutomatedProvider` (`auto`; GS-4 stub;
injectable `::Labeler`, in-process `jobs` store, `::warn_if_same_lineage` ORACLE RULE ? only
warns, does not block; default `not_configured` RAISES NotImplementedError). ? get via
`annotation_registry.get('auto', labeler=fn, lineage='...')`; registered as "auto" only
because no-arg ctor succeeds.
170 - `@backend/annotations/__init__.py` ? ? `noqa F401` imports of
file_provider/automated_provider are load-bearing (removing de-registers providers).
171

```

```

172     ### 2.7 Infrastructure (DB) ?
173     - `@backend/infrastructure/database.py::Database` ? SQLite, 4 tables
    (`videos/play_segments/events/candidate_segments`), `PRAGMA user_version` migration ladder in
    `::init_db` (currently =1, `version<1` creates `candidate_segments`). `::get_connection` (?
    re-issues `PRAGMA foreign_keys=ON` per connection ? else CASCADE/SET NULL silently skipped).
    Methods: `add_video`, `add_play_segment`, `add_event`, `add_candidate_segment`, `link_candidate_to_s
    egment`, `query_candidate_segments(detector=, only_unlabeled=)`, `query_play_segments`, `get_video`,
    `clear_video_data`. ? candidate_segments is the detector-agnostic fine-tuning extension point
    (common fields columns, detector-specifics in `stats`/`detection_params` JSON; index
    `idx_candidates_video_detector`). ? `add_video` is `INSERT OR REPLACE` -> cascade-DELETES
    dependent segments on re-ingest; events column is `type` not `event_type`; `query_play_segments`
    filters use truthiness so `duration_min=0`/empty-string = "no filter"; write helpers swallow
    exceptions.
174
175     ### 2.8 Reporting / observability ? (per-video, soft-dep `obsreport`)
176     - `@backend/reporting/__init__.py` ? `::emit_indexer_report(*, db, video_id, video_path,
    config, val_results, val_context=None, segmenter_requested=, segmenter_actual=,
    was_reingest=False, segmentation_ran=True)` ? single entrypoint, keyword-only. Normalizes typed
    config (`config.model_dump(...)`) if `hasattr model_dump`; short-circuits to None if
    unavailable/disabled; pulls play_segments/candidates from `db` (read-only
    `query_play_segments(vid, {})`/`query_candidate_segments(vid)`); delegates to
    `harvest.record_run`; writes via `rec.write(output_dir=config["report"]["output_dir"])`. ? wraps
    everything in bare `except Exception: return None` ? **NEVER raises** (designed for `main.py`'s
    `finally:`); writes `<video>.report.json`/`.report.md`/`.summary.md` next to video.
    `::OBSREPORT_AVAILABLE` ? soft-dep gate (True only if `import obsreport` AND `from . import
    harvest` both succeed). `::report_enabled(config)` reads `config["report"]["enabled"]` default
    True.
177     - `@backend/reporting/harvest.py` ? records raw FACTS into `obsreport.RunRecorder` (the
    footgun RULES live in spec.yaml, not here). `::record_run(...)` ? top-level assembler (does NOT
    write); builds RunRecorder, reads `config_default = config["indexing"]["default_segmenter"]`,
    runs 4 stage builders + `_highlights`, `rec.finalize()`.
    `::AI_SEGMENTERS={yolo_hybrid, tracknet_hybrid, wasb_hybrid, gemini}` ? (add new AI segmenters
    here). `::video_meta_from_context` (derives VideoMeta from `val_context['ffprobe_meta']`; now
    also `audio_present` for Input Quality reports extension; $ `fps_source` ?
    {unknown, probed, fallback}), `::validation_stage`, `::segmentation_stage` ($ `details.{segment
    er_requested, segmenter_actual, config_default, matches_config_default, segmenter_explicitly_request
    ed, was_reingest, detector, metadata_source}`; ? marks `motion` output "synthetic/heuristic"),
    `::ai_stage` ($ `details.{ai_based, provider, model, api_key_present}`; metric
    `confirmed_segments`), `::highlights` (coverage_pct). ? `record_run` fps fallback:
    `fps_source=='unknown'` + ran -> promotes to "fallback", forces fps=30.0 (mirrors runners).
    `::SPEC_PATH`/`::load_spec` resolve `spec.yaml` next to module. (Soft import of obsreport so
    pure helpers like video_meta are importable for tests/coverage even without the dep.)
178     - `@backend/reporting/spec.yaml` ? $ declarative report layout + footgun engine consumed
    by obsreport. `::customer_verdict` (pass/pass_with_warnings/fail messages). `::sections`
    (ordered `{id, title, kind, audience?{both, customer, technical}}`). ? `::rules` ? each
    `{code, severity, when[{path, op, value}], message, faq_ref, customer_message?}`; `when` clauses AND'd;
    `path` dotted-navigates the recorder JSON. ? `path` strings are a HARD coupling to harvest's
    stage/metric/detail names ? renaming a detail in harvest.py silently breaks the matching rule.
    Rules: `silent_provider_noop` (error: no api key + 0 segs), `ai_empty_vs_no_rallies` (warn),
    `fps_fallback_used` (warn), `synthetic_motion_metadata` (warn), `segmenter_divergence` (info),
    `reingest_replaced_prior` (info), `validation_failed` (error: halts indexing).
179
180     ---
181
182     ## 3. DATA CONTRACTS (canonical shapes ? one place)
183
184     $ **Trajectory CSV** (produced by tracknet/wasb predict, consumed by
    `parse_trajectory_csv`):
185     `Frame, Visibility, X, Y` ? header+column order fixed; `Visibility==1`=visible; X, Y pixel
    coords.
186     -> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).
187
188     $ **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo
    `_extract_action_windows_from_chunk`):
189     `{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,

```

```

mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]]`.
190
191     $ **candidate_segments DB row** (`db.add_candidate_segment` /
`query_candidate_segments`):
192     `{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL,
duration:REAL, confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count},
detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}`
(stats/detection_params deserialized on read).
193     `detection_params` keys by detector:
wasb=`{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}`; tracknet
adds `queue_length`; yolo=`{velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`.
194
195     $ **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
196     `{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server,
receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count,
shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare
`model_name`; motion adds random `intensity/avg_speed_kmh`.
197
198     $ **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time,
end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start;
`_candidate_id` stamped internally).
199
200     $ **GT / golden CSVs** (TWO conventions ? do not conflate):
201     - generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal
or MM:SS/HH:MM:SS; RAISES on bad row.
202     - golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
203
204     $ **det_raw.jsonl** (one window/line): `{ "w":<window_index>,
"f":[[frame_id,[[x,y,score,scale],...]], ...]}`. Lines MUST be contiguous (line N has `w==N`);
first violation/JSONDecodeError -> truncate-heal.
205     $ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
206     $ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv,
...}]}`; ? DB key = file MD5, manifest `video_id` is a label.
207
208     $ **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP),
unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives
,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.
209
210     $ **WSL detector candidate** (in-WSL, model<->tracker): `{ 'xy':np.array([x,y]),
'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST
rebuild `xy` as np.array (tracker does `np.linalg.norm`).
211
212     $ **HitEvent** (audio, `pipeline/audio/hit_detector`): `{t:float(sec), strength:float}`
? onset-envelope peak.
213
214     $ **observability report** (`reporting/harvest.record_run` -> `RunRecorder`): stages
`validation/segmentation/ai_handoff` + highlights + verdict ? {pass,pass_with_warnings,fail};
flags `{code,faq_ref}`; `video_meta.fps_source` ? {probed,fallback}`. ? stage/metric/detail names
are coupled to `spec.yaml::rules[.when[.path]` ? pairs that MUST stay in sync:
`ai_handoff.details.{api_key_present,ai_based}`, `segmentation.metrics.segment_count`, `segmenta
tion.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinges
t}`, `validation.status`, `video_meta.fps_source`.
215
216     ---
217
218     ## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ?
claims below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
219     - Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra

```

```

`compose(config_name='eval', overrides=[dataset=<sport>, model=wasb,
detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is
single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
220 - `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH),
ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read
from cfg["model"] at runtime in wasb_infer.run.)
221 - `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats)` -> (results,
hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.
(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
222 - `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes
every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()`
resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls
`build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y`
? consistent.)
223 - `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh,
transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)
224 - Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-
data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial,
avoided.
225
226 ---
227
228 ## 5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified
here)
229 - Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-
encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest
tracks the exact file used.
230 - `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv`
(indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` +
`@run_phase3_eval.py`.
231 - Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) ->
consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.
232
233 ---
234
235 ## 6. GOTCHAS (cross-cutting footguns)
236 - **MD5 keying:** `video_id` = MD5 of WHOLE file via the single
`@backend/utils/hashing.py::compute_video_id` helper (64KB chunks), imported by main + all
run_*.py. Manifest `video_id` is a label, NOT the DB key.
237 - **Config access:** `backend/main.py` and all four `run_*.py` load via
`backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains
for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in
`main.py` that only fires if `global_config/.indexing` is `None` (the ASGI-import edge case).
238 - **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `motion`
getattr fallback noted above.
239 - **`AppConfig.get` must return dict sections, not models:** the legacy
`.get("indexing",{}).get(...)` chains everywhere depend on it; the `extra="allow"` model also
silently keeps typo'd config keys (no rejection).
240 - **`output_dir`:** a real field on `OutputConfig` (default `output`); the CLI
`--output-dir` overrides it on the typed model in `main()`, and `compile_highlights` reads the
same `global_config.output.output_dir` ? honored end-to-end.
241 - **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if
`{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op. `analyze_video` also
returns `[]` on real failure -> empty != "no rallies". The reporting
`silent_provider_noop`/`ai_empty_vs_no_rallies` rules exist precisely to disambiguate this in
the report.
242 - **Reporting never breaks a run:** `emit_indexer_report` swallows all exceptions
(designed for `main.py` `finally:`) and is emitted even on validation failure. `obsreport` is a
SOFT dep ? absent -> no-op.
243 - **spec.yaml <-> harvest.py coupling:** rule `path` strings dotted-navigate harvest's

```

```

stage/metric/detail names; rename a detail in harvest.py and the matching rule silently stops
firing. Add rules in spec.yaml, not code.
244     - **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner
fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes
`59.94/1920`. The report's `fps_fallback_used` flag surfaces this.
245     - **WSL `/mnt` can't see Google Drive / network mounts;** stage video into native WSL fs
(`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
246     - **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on
PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg +
`--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo
modules).
247     - **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-
reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
248     - **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST
exist in `output/<db_name>` before any eval/regression.
249     - **Validator ordering coupling:** exec order is Format -> ResolutionFPS ->
PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless
`format_check` ran earlier (shared `context['ffprobe_meta']`). PTS 10.0s gap + scene-cut 0.6
correlation are NOT config-overridable.
250     - **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-
deletes its segments. Use `clear_video_data` deliberately. `_get_connection` MUST re-issue
`PRAGMA foreign_keys=ON` per connection.
251     - **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to
P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs
tracknet's `is None` check.
252     - **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs
`backend.annotations` (provider registry). regression.py imports the latter.
253     - **`calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared
API** for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-
CSV schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by
`distill_local`; `harness.get_predictions` reused by `regression.regress`.
254     - **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`,
slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
255     - **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.

```

5. assistant (2026-06-09T11:25:51.415Z)

This is a rich codebase. Let me read the two most relevant strategy docs (the owned-model implementation plan and the training study) and the platform architecture doc to understand the existing language posture, then do targeted web research.

6. user (2026-06-09T11:25:52.474Z)

```

1      # Owned-model implementation plan (DRAFT ? for owner review, no code started)
2
3      **Status:** PLAN ONLY. No owned-model implementation begins until the owner greenlights
a specific
4      milestone (per CLAUDE.md). Paid Gemini stays the always-ready primary/fallback
throughout.
5      **Grounded in:** `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md` (the verified
study) +
6      `docs/archives/quality-iterations/2026-06-multivideo-lovo/` (the LOVO + prototype that
de-risk it).
7
8      ## North star & non-negotiables
9      - **Goal:** an OWNED, on-device rally?highlight model; local inference = the privacy
moat. The moat is the
10     consented dataset + eval harness, **not** the weights.
11     - **Guardrails (hard):** Apache-2.0/MIT bases only (Gemma-3 rejected); **never** train
on paid-LLM
12     (Gemini/OpenAI/Anthropic) outputs; exclude minors + separate training opt-in; split
data **by match**.
13     - **Sequencing discipline:** committed next platform slice is the **async job model
(scaffolding)**. Heavy
14     model *training* (Gen-2, cloud) waits for P0?P4 scaffolding + a healthy corpus +

```

```

explicit go. The
15     **corpus + data + compliance foundation is no-regret and can proceed first** ? it's
what makes later
16     training cheap and clean.
17
18     ## Why now is the right moment (today's evidence)
19     - Heuristic cross-video LOVO =  $0.443 \pm 0.279$  (Adarsh 0.722, Kushagra 0.163-ceiling).
The  $\pm 0.279$  is
20     *method* variance ? the unsupervised windowing is footage-fragile even when WASB
recall is excellent
21     (Kushagra: 97% of rallies have shuttle motion, yet unsegmentable).
22     - The learned prototype gets held-out per-frame AUC 0.84?0.88; Kushagra oracle F1
0.578 vs 0.163.
23     ? The rally signal is learnable; the bottleneck is data scale + calibration (n?5),
not method**.
24
25     ---
26
27     ## Phase 0 ? Foundation (start now; no GPU/scaffolding dependency)
28
29     ### M0.1 ? The labeled-corpus spine (the moat) . *owner-greenlight item*
30     - What: one canonical JSONL row per rally: `{video_id (file MD5), rally_ordinal,
sport, start, end,
31     ending_reason, label_type, source(human|pseudo|vendor), annotator/provenance,
consent/training_opt_in,
32     split(train|val|test, assigned BY MATCH), trajectory_ref, labeled_window}`.
33     - Where: authored in `sports-data-collector` (system-of-record; extends PR #13 + the
positioning doc);
34     the indexer only consumes transcoder outputs.
35     - 3 deterministic transcoders (one row ? three views, zero re-labeling):** (1)
heuristic/LogReg
36     candidate labels (already exists: `training.label_candidates`), (2) TAL ActivityNet-
JSON, (3) VLM clip?QA.
37     - Guardrail enforcement: provenance/consent/minors fields gate corpus membership
before transcoding;
38     Gemini labels never enter a training transcoder; the new `eval_partial_labels` window
flows into every view.
39     - One scorer spine: `metrics.match_intervals` for heuristic-labeling, TAL eval, and
the Gen-2 reward.
40     - Acceptance: round-trip test (collector export ? each transcoder ? indexer
loaders); split-by-match
41     enforced; no Gemini-sourced rows in any training view.
42
43     ### M0.2 ? Grow n?2 ? 5 distinct matches . *owner action + my scoring loop*
44     - What: tag ?3 more distinct matches (gBaaddy, TestLarge, + a small Rooru batch).
I generate the
45     trajectory + score each as it lands (the loop we've been running).
46     - "Healthy corpus" bar: ?5 distinct match-sessions with full human labels ? stable
leave-one-match-out.
47     - Acceptance: n?5; per-match + LOVO numbers recorded; corpus diversity
(camera/court/lighting) noted.
48
49     ### M0.3 ? Compliance cleanup (do before it bakes in) . *owner-greenlight item*
50     - (i) Purge Gemini-derived TRAINING labels. The distilled pipeline currently trains
on Gemini silver
51     (`training.label_candidates` over Gemini CSVs) ? this violates the no-paid-LLM-
training rule. Retarget
52     training to human + open-teacher (WASB/own-model) labels only; Gemini stays
eval/teacher-reference.
53     - (ii) WASB checkpoint provenance (C3). The distributed
`wasb_badminton_best.pth.tar` is academic-data-
54     trained (provenance unresolved) ? it propagates to anything trained on its features.
Decide: document
55     as a hard Gen-0 prerequisite and plan ROADMAP Phase-4 "train own WASB weights," or
scope an alternative.

```

```

56     - **Acceptance:** no paid-LLM outputs in any training path; a written C3 decision +
prerequisite flag.
57
58     ### M0.4 ? Gen-0 A/B harness (built here; trains on your GPU box) · *owner-greenlight
item*
59     - **What:** productionize the prototype into a real harness ? feature stack (visibility-
run-length, velocity,
60     acceleration, net-crossing flags, smoothing/windowing) ? **ActionFormer (MIT, 1:1
rally=instance)** and/or
61     **ASFormer (MIT, TAS)** train?eval?**ship-gate**. Reuse `metrics.match_intervals`.
62     - **Ship gate (the honest bar):** held-out per-match F1 must beat the **honest cross-
video heuristic LOVO
63     (~0.44)**, not the fit-on-self 0.73. nanochat-style pass/fail verdict.
64     - **Where it runs:** training/eval on the **GPU/WSL machine** (can't run in this dev
container). Harness +
65     feature stack are authored here; you run the A/B.
66     - **Acceptance:** one-command train?eval?verdict; reproducible; the prototype's AUC
translated to segment F1
67     at n?5; documented A/B (ActionFormer vs ASFormer vs heuristic).
68
69     ---
70
71     ## Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
72     - Run the A/B; if a learned Gen-0 clears the honest LOVO bar with per-video calibration,
**it becomes the
73     rally segmenter** (still fully local, MIT, on the 2070 Super). If not, the finding
(better data /
74     features needed) is itself the next-step signal ? no over-fitting to declare victory.
75     - **Decision gate:** Gen-0 adopted ? more data/features first. Either way, the corpus +
harness persist.
76
77     ## Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + explicit go)
78     - **Gen-1 (Option A):** concatenate fully-local cue channels (audio-onset, pose/court)
onto the same head;
79     fusion in our rally layer. License/cost-vet the pose extractor (open item).
80     - **Gen-2 (Option B):** e2e multimodal VLM on an Apache-2.0 base (**Qwen3-VL-4B**
primary / Qwen2.5-VL-7B
81     fallback / **Gemma 4** end-state once a timestamp recipe is proven), SFT cold-start ?
GRPO/RFT with a
82     temporal-IoU-vs-golden reward. **Cloud training on Modal** (best no-access contract);
base license
83     re-verified per-checkpoint.
84     - **Hard pre-serving gate:** the **$/call crossover benchmark** (self-hosted-VLM vs
Gemini on real workload)
85     + an **on-device latency/throughput** measurement of the quantized base on the 2070
Super ? both currently
86     *unmeasured* and load-bearing. Don't stand up an owned serving stack until these clear
(COMMERCIALIZATION C7).
87
88     ---
89
90     ## Decisions I need from you (to unblock specific milestones)
91     1. **Greenlight which Phase-0 milestone(s) to start in code** (M0.1 corpus spine = my
recommendation; M0.3
92     cleanup; M0.4 harness). M0.2 (tagging) runs regardless.
93     2. **WASB C3 stance** ? accept "document as prerequisite" for now, or prioritize own-
WASB-weights sooner?
94     3. **Ratify "Apache-2.0/MIT"** (the study's approved list includes MIT bases/teachers;
the guardrail text
95     says "Apache-2.0 only").
96     4. **Corpus home** ? confirm the collector is the system-of-record for the spine
(consistent with PR #13).
97
98     ## Open prerequisites (carried from the study's critic ? none block Phase 0)
99     WASB C3 provenance · Gemma-4 license stays AMBER until a raw HF LICENSE check · on-

```

device VLM latency
100 unbenchmarked · real VLM VRAM/token cost on our 1080p/4K-60fps clips · DPA + no-train
for any cloud training ·
101 the owned-vs-Gemini \$/call crossover volume.
102
103 ---
104 *This plan is design-only and reversible. Nothing here starts until you greenlight a
milestone.*
105

7. user (2026-06-09T11:25:52.682Z)

```
1      # Owned-model training study (rally segmentation ? end-to-end multimodal)
2
3      **Date:** 2026-06-08 · **STATUS: DESIGN BLUEPRINT (no implementation until PLATFORM
P0?P4 + healthy corpus; owner sign-off required).**
4
5      > How to train our **OWNED** rally/highlight model **reusing the videos we label**,
aligned to the
6      > Apache-2.0 / no-paid-LLM-training / privacy-local guardrails. Produced by an 11-agent
research
7      > workflow (ground ? 5 web-research dimensions ? adversarial verification of
costs/licenses/privacy ?
8      > synthesis + completeness critic; 35 sources). Empirical results from THIS session
(LOVO + a learned
9      > prototype) are folded in and **resolve the study's biggest open unknown** ? see $"What
we proved today".
10     > Numbers from web sources are 2026 snapshots in a ±15?25% band; **re-quote live before
budgeting.**
11
12     ## Thesis
13
14     Train the owned model in **three staged generations off ONE model-agnostic labeled
corpus** ? a label
15     collected once feeds every generation, so labels are never re-collected. **The moat is
the consented
16     dataset + the eval harness, never the weights.**
17
18     | Gen | When | Approach | Base / model | Compute |
19     |---|---|---|---|---|
20     | **Gen-0** | now ? unblocks at n?5 matches | tiny **temporal action segmentation
(TAS)** head over a hand-built per-frame feature stack from the WASB trajectory we already
produce; per-frame rally(1)/non-rally(0) ? merge into [start,end] | **ASFormer (MIT, ~1.1M
params)** primary; **ActionFormer (MIT)** A/B challenger (overtakes once corpus = hundreds of
matches) | **local RTX 2070 Super**, trains in minutes, $0 |
21     | **Gen-1 / Option A** | 6?12 mo | concatenate more local cue channels onto the SAME
head: trajectory + audio-onset + pose/court serve-gating; fusion in our rally layer, cues
swappable | same ASFormer/ActionFormer, wider input | local + occasional cloud burst (pseudo-
labeling) |
22     | **Gen-2 / Option B** | 12?24+ mo, after scaffolding + healthy corpus | end-to-end
multimodal VLM: (frames+audio+pose+trajectory) ? rally [start,end] (+semantics); SFT cold-start
then **GRPO/RFT with a temporal-IoU-vs-golden reward** | **Qwen3-VL-4B-Instruct** (Apache-2.0,
on-device) primary; **Qwen2.5-VL-7B** fallback; **Gemma 4 E4B/12B-Unified** (Apache-2.0 + native
video+audio) end-state | rented 24?48 GB A100/L40S (does NOT fit 8 GB box) |
23
24     ## What we proved today (de-risks the blueprint)
25
26     The study's **"dominant empirical unknown"** was: *can a low-dim WASB trajectory (?3?4
raw dims) feed a
27     TAS head, given ASFormer's small-data result used 2048-d I3D features?* This session
answered it.
28
29     - **Cross-video LOVO of the current heuristic** (`calibrate_local`, Adarsh-96 +
Kushagra-32, human labels):
30     mean held-out F1 **0.443 ± 0.279** ; Adarsh 0.722, **Kushagra 0.163** (its own ceiling
```

```

? *no* velocity-windowing
31     config does better). **The  $\pm 0.279$  spread is method variance, not config.** ? This also
**corrects the study's
32     "~0.73 bar": that was a fit-on-self single-match number; the honest cross-video
heuristic bar is **~0.44**,
33     which *lowers* the Gen-0 ship gate.
34     - **A learned per-frame prototype** (`backend/eval/rally_seq_proto.py`: 6 hand-built
features ? visibility rate,
35     mean/max shuttle speed, **net-crossing rate**, x/y spread ? into the repo's
dependency-free LogisticRegression,
36     honest LOVO): **per-frame AUC 0.878 (Adarsh) / 0.841 (Kushagra), held-out.** On
Kushagra ? the match the
37     heuristic *cannot* segment (0.163) ? a calibrated threshold reaches **segment F1 0.578
(3.5 $\times$ )**. The naive
38     cross-video threshold gives only 0.049, so **the remaining gap is threshold/operating-
point calibration at n=2,
39     not representation.**
40
41     - **n=3 update (doubles match added ? `LargeTestVideo`, 49 rallies):** the learned
prototype **overtakes the
42     heuristic.** Mean held-out LOVO F1 ? learned **0.377 ? 0.615**, heuristic **0.443 ?
0.544**. Kushagra's learned
43     held-out F1 jumped **0.049 ? 0.417** (one added match closed most of the calibration
gap); the **doubles** match,
44     held out and trained only on the two singles, scored AUC **0.914** / F1 0.729
(generalizes across play type +
45     frame rate). The heuristic still cannot rescue Kushagra at any n (0.151, capped).
**The honest "bar to beat" is
46     thus ~0.54 (n=3), not 0.73 ? and the learned path already clears it on the mean and
keeps rising with n.**
47
48     **Conclusion:** the rally signal IS learnable from the WASB trajectory features, cross-
footage (AUC ~0.85?0.91) ?
49     the heuristic just can't exploit it. This **validates the Gen-0 ASFormer-over-WASB-
features path**, shows the
50     hand-built feature stack the study calls for is the right lever, and confirms the
bottleneck is **data + per-video
51     calibration (n $\geq$ 5 matches), not method.** The prototype is the seed of Gen-0, and the
n=2?n=3 trend (learned mean
52     +0.24, overtaking the heuristic) is the empirical green light.
53
54     ## Platform ranking (privacy + cost lens; the fine-tune is a sub-24h, $10?50 job ?
privacy/ergonomics dominate)
55
56     1. **Modal ? PRIMARY.** Strongest *written* no-access contract (verbatim: "Modal will
never access? the inputs/outputs to your Functions, or any data you store"), ?7-day TTL,
SOC2-II, HIPAA-BAA; per-second scale-to-zero matches retrain-on-each-batch. A100 80 GB
~$2.50/hr, H100 ~$3.95/hr (corrected down from cited). Privacy is the load-bearing reason, not
price.
57     2. **RunPod (Secure Cloud tier ONLY) ? FALLBACK / Option-B scale-up.** Cheapest vetted-
DC self-serve; A100 80 GB ~$1.39?1.49/hr. Community/P2P tier is UNVETTED ? **never** touches
consented video. Confirm AES-256/DPA at booking.
58     3. **Vertex AI / GCP ? ON BENCH.** Only strong-privacy option with an India region, but
it's **capacity-gated** (asia-south1 high-end GPU is invitation-only, H100-class A3 not A100)
and **tuning Gemini yields NO portable on-device weights** ? only tuning an Apache-2.0 base in
Model Garden does. Use only if India residency-at-rest is mandated.
59     4. **Self-hosted RTX 2070 Super 8 GB** ? Gen-0 training + ALL inference + CI only.
**Cannot** QLoRA a 7B video VLM (~20?24 GB). Best privacy (nothing leaves the box).
60     5. **Fireworks** (managed, contractual no-train + ZDR) usable; **Together BLOCKED**
until a signed DPA + explicit no-train clause. AWS SageMaker / Azure ML = heavy for a solo
founder; India-residency only.
61
62     ## Label reuse ? one corpus, three transcoders (the actual moat)
63
64     Promote the rally CSVs from eval-only files to **one model-agnostic corpus**: JSONL, one

```

```

row per
65     `(video_id, rally_ordinal)` = `{start, end, ending_reason, sport, source
(human|pseudo|vendor),
66     annotator/provenance, consent/opt-in, label_type, split (BY MATCH not clip),
trajectory_ref}`. The
67     provenance/consent fields are the **enforcement point** for the legal guardrails. Three
deterministic
68     transcoders read it, zero re-labeling:
69     1. **Heuristic/LogReg view** ? already exists
(`backend/eval/training.py::label_candidates` via `metrics.match_intervals`).
70     2. **TAL view** ? ActivityNet JSON (`segment:[start,end]`, `label:'rally'`); our seconds
map 1:1.
71     3. **VLM clip?QA view** ? "list every rally [start,end] in this clip"; `ending_reason` ?
a second QA task.
72
73     **One scorer spine:** reuse `metrics.match_intervals` for heuristic-labeling, TAL eval,
AND the Gen-2 RFT
74     reward ? no metric drift. **Leakage guard:** split **by match, never by clip**.
**Amplification (all
75     guardrail-clean):** open-teacher pseudo-labels (WASB/own-model, confidence-gated ? never
paid-LLM),
76     boundary-preserving temporal augmentation, active learning to spend the Roora budget on
highest-uncertainty
77     matches. ? **The new partial-label windowing**
(`eval_partial_labels.restrict_to_window`) must flow into the
78     TAL/VLM views so prefix-labeled videos aren't mis-scored (a corpus that's partially
tagged is the norm).
79
80     ## Licensing & privacy guardrails (all respected; verify per-checkpoint)
81
82     - **Bases: Apache-2.0/MIT only.** APPROVED: Qwen2.5-VL-7B/32B (Apache-2.0),
Qwen3-VL-4B/8B (Apache-2.0),
83     Gemma 4 E2B/E4B/12B-Unified (Apache-2.0 + native video/audio), InternVL3 ?38B
(MIT/Apache), ASFormer/
84     ActionFormer/TriDet (MIT). **REJECTED:** Gemma 3 (viral Gemma Terms + PUP),
Qwen2.5-VL-3B (NC research),
85     Qwen-72B/InternVL3-78B (100M-MAU cap), Llama Vision (700M-MAU), DAMO VideoLLaMA3
weights (NC + paid-LLM-trained),
86     TimeChat/VTimeLLM/Momentor (NC/no-license = method-reference only), VideoMAE weights
(CC-BY-NC), `yabufarha/ms-tcn`
87     (Commons Clause ? use `sj-li/MS-TCN2` MIT). ? **Gemma 4 size names =
E2B/E4B/26B-A4B/31B/12B-Unified ? there is no plain "4B".**
88     - **No paid-LLM training data** ? Gemini/OpenAI/Anthropic = inference/eval/teacher-
reference only, **never** a
89     training signal. Gemini silver labels stay out of every transcoder.
90     - **Exclude minors by default + separate training opt-in;** per-user adapters trained
only on that user's
91     consented video (deletion = drop the adapter). Enforced at the corpus-membership layer
before transcoding.
92     - **Privacy lever = local inference** (Gen-0 ~1.1M params; Gen-2 4B quantized). Any
cloud *training* needs a
93     signed DPA + no-train assurance; route only owned/consented footage, never user
uploads.
94
95     ## Cost (spend on labels, not GPUs)
96
97     - **Gen-0:** ~$0 (local 2070 Super; trains in minutes; ~4.5 GB VRAM).
98     - **Gen-2 LoRA:** ~$5?40/run on a rented 24?48 GB GPU (RunPod cheapest; **VRAM/token
cost on real 1080p/60fps
99     clips is UNVERIFIED ? 24 GB is a floor, re-benchmark**). Managed (Fireworks/Together)
~$2?30 *if* text-priced
100     (video tokens can balloon). RFT = compute-heavy (multiple rollouts/sample).
101     - **Inference crossover (the unanchored number):** self-hosted 7B beats Gemini
(~$0.05/10-min clip) **only**
102     above sustained QA volume; below it, **paid Gemini is cheaper** ? paid-primary now,

```

owned-primary later.

```

103     **Benchmark $/call before committing** (COMMERCIALIZATION C7).
104
105     ## Open risks & critic caveats (read before acting)
106
107     - **?? WASB checkpoint provenance (C3) propagates.** The study calls WASB "MIT," but
that's the *code*; the
108     distributed `wasb_badminton_best.pth.tar` is **academic-data-trained, provenance
unresolved** (COMMERCIALIZATION
109     C3, still open). Every Gen-0/1 plan uses WASB features as input AND as a pseudo-label
teacher ? a "clean" owned
110     model trained on provenance-unclear features **does not launder the provenance.**
ROADMAP Phase-4 "train own WASB
111     weights" is a **prerequisite, not an aside.**
112     - **Gemma 4 "clean + native-video" rests on post-cutoff secondary sources** ? keep AMBER
until a raw HF LICENSE
113     revision is verified, given the Gemma-3 rejection. Its timestamp-emission recipe is
less documented than Qwen's.
114     - **Existing distilled model was trained on Gemini silver labels** ? the (correct) eval-
only policy now **forbids
115     this; that pipeline must be purged of Gemini-derived training labels** to comply.
116     - **Policy ratifications needed:** guardrail literally says "Apache-2.0 only" but the
approved list includes MIT ?
117     ratify "Apache-2.0/MIT". The MAU-cap rejections (72B/78B) may be over-conservative at
our scale.
118     - **On-device privacy claim is unbenchmarked** ? no tokens/sec for a quantized 4B VLM on
8 GB Turing at
119     1080p/60fps. **Measure before relying on it.**
120     - **Amplification is load-bearing for Gen-2** ? 128 raw rallies can't reach the ~5k?20k
clip?QA band alone; if
121     augmentation + QA-multiplexing + pseudo-labels underdeliver, Option B is under-fed.
Amateur 60fps may need more
122     labels than broadcast-trained literature implies.
123     - **Dropped option worth keeping:** InternVideo2 encoder (MIT) + light grounding head ?
the *only* data-realistic
124     owned-video experiment available at n?128 today. And: a **one-time ~$1?2k 24 GB local
card** (e.g. used 3090)
125     would preserve the privacy lever for *training* and dissolve most of the cloud-DPA
analysis ? uncoded here.
126
127     ## Open questions (need the GPU/WSL machine-side suite ? not runnable in this dev
container)
128
129     1. Does ASFormer/ActionFormer over the WASB feature stack beat the **honest ~0.44 LOV0**
bar at n?5 matches?
130     (Today's prototype says the per-frame AUC is there ? 0.84 ? so this is now *likely*;
confirm at segment level.)
131     2. Real VRAM + token cost of fine-tuning Qwen3-VL-4B / Qwen2.5-VL-7B on our actual
clip?QA frame counts?
132     3. Working Gemma-4 timestamp-emission recipe + exact checkpoint (E4B vs 12B-Unified)?
133     4. Measured self-hosted-7B $/call + latency vs Gemini on the target workload (the owned-
model ROI crossover)?
134     5. Will Together/Lambda/Azure sign a DPA with explicit no-train for consented video,
India-configurable?
135     6. Realistic clip?QA yield from 128 rallies via augmentation + QA-multiplexing + pseudo-
labels?
136
137     ## Sequencing (owner directive, non-negotiable)
138
139     This is a **design blueprint only.** It unblocks PLATFORM **P5** but starts **no
implementation** until P0?P4
140     scaffolding (async jobs, storage/compute backends) is rock-solid AND the golden corpus
is healthy. Paid Gemini
141     stays the always-ready primary/fallback. The only items safe to act on now ? **and even
these need owner

```

```

142     sign-off before code** ? are (a) the model-agnostic corpus format and (b) growing n?2 ?
n?5 matches (a small
143     Roora batch), which unblocks honest tuning immediately and is the **highest-ROI spend
(labels, not GPUs).**
144
145     *Full agent transcript + 35 sources: workflow `wf_49aab888-e10` (this session).*
146

```

8. assistant (2026-06-09T11:25:55.721Z)

Now let me read the platform architecture doc, which is where the Python/FastAPI vs Go/Rust hot-path question lives, plus the commercialization doc for cost/serving context.

9. user (2026-06-09T11:25:57.214Z)

```

1     # Platform Architecture ? Multi-User "Assistant Coach"
2
3     **Status:** ? Strategy / research ? for review. No code yet.
4     **Date:** 2026-06-07 . **Owner:** avidullu
5
6     **Purpose.** Map the path from today's **single-user, local** tool to a **multi-tenant
7     "assistant coach" platform**: users upload videos, get highlights **stored and
8     browsable by history**, and choose **where their data lives** (Google Drive / cloud
9     bucket / local) and **where the compute runs** (our GPU / their VM). The organizing
10    principle is to **decouple the layers** ? identity, ingestion, storage, orchestration,
11    execution/compute, the detection+highlight pipeline, and data-serving ? behind small
12    **pluggable interfaces**, reusing the registry pattern the codebase already uses for
13    segmenters/providers/validators/sports/annotations.
14
15    **Product framing (the north star *and* the v1 line).** This is an **assistant coach**,
16    not just a clipper ? but the strategy is **phased and unambiguous about scope**:
17    - **v1 (concrete, near-term):** **rally detection ? highlight-reel compilation ? per-
user
18    history ? a coach/user correction loop that feeds the golden set** ("assistant-coach
19    *lite"). Essentially today's pipeline made multi-user ? this is the real v1
deliverable.
20    - **Mid-term:** **rich rally/shot semantic metadata** (shot type, rally outcome) and the
21    **owned video-QA model** ($5A; $6 P4?P5).
22    - **North-star vision (strictly future ? *not* v1):** **gait / biomechanics, per-athlete
23    identifying templates, and deep coaching feedback under a supervising coach** ($6 P6).
24    Inspiring, but explicitly deferred ? and the part that raises the **biometric**
privacy
25    bar ($7).
26
27    The buyer is the **academy/coach** (beachhead in
28    [`PMF_MARKET_RESEARCH.md`](PMF_MARKET_RESEARCH.md)); **power users self-serve**. Two
29    consequences ripple through the design: (1) the data model keeps an **optional coach ?
30    athlete overlay over a generic core** ($4.1) ? not baked in; (2) gait/biometrics is
31    **special-category data**, gated and deferred ($7).
32
33    > **Competitive note (validated by research):** none of Veo / Hudl / SwingVision /
34    > Spiideo offer true BYO-storage or BYO-compute ? they're vendor-cloud SaaS
35    > (SwingVision uniquely runs *edge* compute on the user's iPhone). So **"your video,
36    > your storage, your compute" is a genuine differentiator for privacy-sensitive
37    > academies ? and SwingVision proves user-side compute is commercially viable. All four
38    > share the same **teams ? athletes ? clips** data model, which we adopt ($4).
39
40    > Scope: this is a **direction-setting doc**. It does not start implementation; the
41    > first infra slice (an async job model) is the on-hold #16 in
42    > [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md). Commercial constraints
come
43    > from [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) (C4 paid-LLM no-train; C12 SaaS
44    > foundations) and `archives/MONETIZATION_AUDIT.md`.
45
46    ## 1. Current state & gaps (what we're evolving from)

```

```

47
48 | Concern | Today | Gap for multi-user |
49 |---|---|---|
50 | Identity / tenancy | none | no users, orgs, auth, or isolation |
51 | Storage | local `filepath` string; output to `output_dir` | no abstraction; can't BYO
Drive/bucket; raw video must be on the box |
52 | Data model | `videos` keyed by file **MD5**, `play_segments`, `events`,
`candidate_segments` (SQLite) | no owner/athlete link; **highlights aren't a first-class
persisted entity** (compile writes a file and returns a path); no per-user history |
53 | Execution | pipeline runs **in-process, synchronously** in `/api/process` | blocks;
can't dispatch to a user's VM/GPU; no job status |
54 | Pipeline | **already pluggable** (segmenter/provider/validator/sport/annotation
registries) | reuse as-is ? this layer is the model for the rest |
55 | API | `/api/process`, `/api/query`, `/api/compile`, `/api/config`; no auth | needs
auth, per-tenant scoping, async job + history endpoints |
56
57 Grounding: `backend/infrastructure/database.py` (schema + `PRAGMA user_version`
58 migration ladder, v1), `backend/main.py` (endpoints, sync flow), the registry pattern
59 in `backend/pipeline/segmenters/base.py::SegmenterRegistry` and
60 `backend/providers/base.py`.
61
62 ## 2. Target layered architecture
63
64 Decouple into layers with a clean **control-plane vs data-plane** split ? the
65 orchestrator coordinates *metadata and jobs*; the heavy *bytes and GPU work* can live
66 elsewhere (even in the user's own cloud).
67
68 ...
69
70 CONTROL      ?????????????????????????????????????????????????????????
71 PLANE        ? Presentation / API (auth, per-tenant) ?
72              ? Identity & Tenancy (org?coach?athlete?user) ?
73              ? Orchestration / Jobs (submit?status?result) ?
74              ?????????????????????????????????????????????????????????
75              ? dispatch ? metadata
76              ?????????????????????????????????????????????????????????
77 DATA        ? Execution / Compute ? ? Data / Serving ?
78 PLANE        ? (local | BYO-VM | ? ? (per-tenant DB: ?
79              ? hosted GPU) ? ? videos, segments, ?
80              ? runs the Pipeline: ? ? highlights, jobs, ?
81              ? detect ? highlight ? ? history) ?
82              ?????????????????????????????????????????????????????????
83              ? read input / write artifacts
84              ?????????????????????????????????????????????????????????
85              ? Storage (local upload | Google Drive | S3 | ?
86              ? GCS | Azure) ? raw video + highlight outputs ?
87              ?????????????????????????????????????????????????????????
88
89 ...
90
91 The two **new pluggable seams** are **Storage** and **Compute/Execution**; everything
92 else either exists (pipeline) or is net-new infra (identity, orchestration, serving). A
93 **model/training layer** ? the owned generative video-QA model ($5A) ? sits atop these,
94 fed by Storage (training corpus), Data (labels), and Compute (training + inference), and
95 is delivered through the existing **AI-provider registry with a paid fallback**.
96
97 ## 3. Pluggable abstractions (extend the existing registry pattern)
98
99 The codebase already proves the model: an `ABC` + a `Registry` singleton with
100 `register/get/list_available` (e.g. `SegmenterRegistry`). Add two more registries with
101 the same shape so they're plug-and-play and testable in isolation.
102
103 ### 3.1 `StorageBackend` registry
104 Replace the bare `video_path: str` that flows through the pipeline with a
105 **`StorageRef`** (a backend id + a key/URI), resolved by a backend.
106
107 ```python

```

```

106 class StorageBackend(ABC):
107     name: str
108     def open_read(self, ref: StorageRef) -> BinaryIO: ... # stream large video
109     def download_to_local(self, ref: StorageRef, dst: str): ... # for ffmpeg/cv2/WSL
110     def upload(self, local_path: str, ref: StorageRef): ... # persist
highlight/artifacts
111     def signed_url(self, ref: StorageRef, ttl: int) -> str: ... # hand to LLM / browser
112     def stat(self, ref: StorageRef) -> StorageStat: ...
113     def list(self, prefix: StorageRef) -> list[StorageRef]: ...
114     ...
115
116 - Recommended implementation: build the backend over fspec (one matrix ?
117   local / S3 (s3fs) / GCS (gcsfs) / Azure (adlfs) + Google Drive via gdrivefs,
118   Dropbox via dropboxdrivefs), reaching for smart_open as the robust large-MP4
119   byte-streaming primitive. Storage becomes one more registry. (Treat Drive/Dropbox as
120   add-ons ? less battle-tested than s3fs/gcsfs; cover with integration tests.)
121 - Backends: local (upload dir), gdrive, s3, gcs, azure. BYO-storage:
122   the tenant registers their own bucket/Drive; raw video never leaves their account.
123 - Auth ? never persist users' long-lived keys. For buckets, have the user create an
124   IAM role you AssumeRole into, scoped to one bucket/prefix, and mint presigned
125   URLs so raw video moves browser ? the user's bucket directly, never through our
126   servers. For Drive, OAuth with the narrow drive.file scope; for academies, a
127   service account with domain-wide delegation. Store any per-tenant creds encrypted.
128 - Why it matters for the pipeline: ffmpeg/OpenCV/WSL need a local file, while
the
129   LLM provider can take a signed URL. The backend exposes both download_to_local
130   (for compute) and signed_url (for the provider/browser), so the pipeline stops
caring
131   where the bytes live.
132 - Migration path: introduce StorageRef alongside the current local path; a local
133   backend wraps today's behavior so nothing breaks; adapters land incrementally.
134
135 ### 3.2 ComputeBackend / ExecutionBackend registry
136 Where the pipeline actually runs for a given job.
137
138 - local ? in-process / local subprocess (today's behavior; dev + self-host).
139 - hosted ? our GPU VM pool (the SaaS default).
140 - byo_worker ? the tenant runs a pull-based worker (a small agent) on their
own
141   VM/GPU. Critical security property: the worker pulls jobs scoped to its tenant
142   from the control plane, pulls input from the tenant's own storage, runs the
143   pipeline, and writes artifacts + results back ? so the orchestrator never holds the
144   user's cloud credentials and needs no inbound access to their network. This
145   mirrors the self-hosted CI runner pattern (GitHub Actions runners use a 50s
146   outbound-only HTTPS long-poll). This pull-based runner is the cheapest, most
147   credential-safe v1 for BYO-compute.
148 - For users who want the platform to provision GPUs in their own cloud account on
149   demand (an academy with an AWS account but no standing GPU box), add SkyPilot
150   (BYOC: everything launches in the user's accounts/VPCs; creds stay with them).
151   Modal is the managed fallback for users wanting zero infra (accepting that raw
152   video then flows through Modal).
153 - Operational / support burden (a real adoption risk). For the primary beachhead
?
154   Indian badminton academies/coaches ? asking them to run and maintain a worker on
155   their own VM/GPU is a meaningful ongoing load (updates, monitoring, firewall/NAT, the
156   "it stopped after our IT change" ticket). So: default academy users to hosted,
and
157   treat byo_worker/SkyPilot as a power-user / enterprise / privacy-sensitive tier,
158   not the academy default. If/when BYO ships, plan for auto-updating, self-healing
159   agents + health heartbeats + clear worker status in the UI, and budget the support
cost.
160 - Middle tier to consider ? "zero-infra BYO": a managed-but-isolated path where the
161   storage stays in the user's bucket (signed URLs) while compute runs on our
GPUs
162   in a strictly isolated, per-tenant, ephemeral sandbox (nothing persists our side).

```

```

163     This gives the BYO *privacy* benefit (raw video never persists on our infra)
**without**
164     the user running a worker ? a pragmatic default between full-hosted and full-BYO.
165
166     The pipeline code is unchanged; only *where it's invoked* and *how it gets bytes* vary.
167
168     ### 3.3 Reuse, don't reinvent
169     The existing `segmenter` / `provider` / `validator` / `sport` / `annotation` registries
170     are the proven precedent and stay as the **pipeline** layer. The `annotation` registry +
171     `candidate_segments` table are also the natural seam for the **coach-correction / golden
172     -set** loop in the assistant-coach phase (§6, P4).
173
174     ## 4. Multi-tenant data model & serving layer
175
176     ### 4.1 Entities ? generic core, with the coach model as an optional overlay
177     The foundational schema is deliberately **domain-neutral** so we don't overfit to the
178     coach mindset ? the coach/athlete framing is a **market-fit hypothesis that will
iterate**
179     (see C13 / PMF), not a load-bearing assumption. **Generic core:**
180     ```
181     User ??< VideoCollection ??< Video ??< Highlight ??< HighlightMetadata
182           ??< PlaySegment ??< Event
183     Video / Highlight ??< Job (processing record: status, compute backend, timings)
184     ```
185     - **Generic & re-targetable:** a `VideoCollection` groups a user's videos (a season, an
186       event, a "folder") with no assumption of a coach or academy ? a power user, a coach,
or a
187       parent are all just `User`s with collections.
188     - **Highlights are first-class** (today compile only writes a file): a `highlights`
table
189       (segment ids + output `StorageRef` + params + created_at) powers "see past videos and
190       highlights by history," and **`HighlightMetadata`** carries the **rich semantic tags**
191       (shot type, rally outcome, etc. ? see §6 metadata stage) plus titles / ratings /
notes.
192     - **`video_id` stays the content MD5** (dedup within a tenant), scoped by `tenant_id`.
193     - **Coach overlay (optional, additive):** for the academy/coach GTM, layer
194       `Org/Academy ? Coach ? Athlete` *on top* (e.g. a `Membership`/role table + an
`athlete_id`
195       tag on collections/videos) **without** baking it into the core tables ? so the schema
196       stays usable for solo power users and the coach model can evolve, or be dropped, as
PMF
197       dictates.
198     - **History/serving API:** list collections/videos/highlights per user (and, with the
199       overlay, by athlete/coach/date); **query highlights by `HighlightMetadata`** (e.g.
shot
200       type / outcome); re-stitch from stored segments without reprocessing.
201
202     ### 4.2 Tenancy isolation ? options & recommendation
203     | Model | Isolation | Ops cost | Best for |
204     |---|---|---|---|
205     | **Shared DB + `tenant_id`** (row-level, ideally Postgres **RLS**) | logical | low |
**Hosted SaaS, many small tenants** ? *recommended default* |
206     | Schema-per-tenant | medium | medium | tens?hundreds of mid tenants |
207     | **DB-per-tenant** (incl. **SQLite-per-tenant**) | strong/physical | higher at scale |
**Self-hosted single-academy**, or high-isolation enterprise |
208
209     **Recommendation:** for the hosted product, **migrate SQLite ? Postgres with shared-DB +
210     `tenant_id` + Row-Level Security**; keep **SQLite-per-tenant** as the **self-hosted /
211     single-academy** deployment (it reuses today's code almost verbatim). Either way,
212     **abstract DB access behind a repository layer now** so the engine is swappable, and
213     graduate the `PRAGMA user_version` migration discipline to **Alembic** under Postgres.
214
215     ## 5. Orchestration / async jobs
216
217     This is the serving layer that makes multi-user real. It is the **multi-user superset of

```

```

218     [ `DEFERRED_HARDENING_PLAN.md` ] (DEFERRED_HARDENING_PLAN.md) #16** (async job model) ?
build
219     that first, single-tenant, then generalize.
220
221     - **Job record** ( `jobs` table): `id, tenant_id, video_id, type (index|compile|analyze),
222       status (queued|running|done|failed), progress, compute_backend, result_ref, error,
223       timestamps`. `~/api/process` returns **202 + job_id** ; `~/api/jobs/{id}` polls.
224     - **Idempotency** keyed on `(tenant_id, video_id, pipeline_version)` ? reuse the
225       content-hash discipline already in `compute_video_id`.
226     - **Dispatch:** the orchestrator enqueues; a worker (hosted or `byo_worker`) claims jobs
227       for its tenant, resolves input via the tenant's `StorageBackend`, runs the pipeline,
228       writes artifacts + DB rows. **Observability reports** (already per-video) become the
229       job artifact surfaced in history.
230     - **Queue/worker backend ? a decision to prototype in #16, *not* a default here.** The
231       *requirements* are firm ? **idempotency**, **resumability** (a 40-min GPU job mustn't
232       restart on a worker blip), **progress states** ? which rule out fire-and-forget
Celery/RQ
233     for the long jobs. The *choice* should be made by prototyping in the
234     [ `DEFERRED_HARDENING_PLAN.md` ] (DEFERRED_HARDENING_PLAN.md) #16 slice, weighing:
235     - **Start simplest:** **Arq** (async-native, natural FastAPI fit) or even a **plain
236       Postgres-backed queue + worker** ? likely sufficient for the *first* single-tenant
237       async slice, with the lowest operational complexity and the easiest debugging.
238     - **Durable-execution upgrade if/when needed:** **Hatchet** (Postgres-backed, per-step
239       retries/timeouts/concurrency; pairs neatly with the pull-based worker) ? attractive
240       because it adds **no new datastore** once we're on Postgres, but it *is* a new
241       dependency + operational/debugging surface; adopt only when long-job resumability/
242       observability demands it.
243     - **Heaviest:** **Temporal** ? most mature durable workflows, operationally heavy;
244       reserve for genuinely complex multi-stage pipelines at scale.
245
246     Record the decision criteria in #16: operational complexity, maturity, **debugging
247     experience for long GPU jobs**, and whether durability is worth a new dependency.
248     **Bias: start with the simplest thing that gives retries + status; graduate only on
249     evidence.**
250
251     ## 5A. Owned generative model (the video-QA assistant) & the data flywheel
252
253     The north-star capability: an **owned, self-hostable generative video model** ?
254     fine-tuned on sports video (including **consented, user-contributed** footage) ? that
255     **answers questions about rallies, shots, and technique**. This is the conversational
256     brain of the "assistant coach." Crucially, it does **not** replace the known stack; it
257     is introduced **behind the same abstraction, with the paid API as an always-ready
258     fallback**.
259
260     > **The moat is *not* the model.** Anyone can download Qwen-VL/Gemma. The defensible
261     > assets are ** (a) the consented, badminton-specific video?QA dataset** and ** (b) the
262     > eval harness**. Build and message *those* ? not "we own an LLM."
263
264     > **? Committed guardrail (legal, non-negotiable).** We respect the licensing findings
in
265     > full. **Paid frontier models (Gemini / OpenAI / Anthropic) are used for *inference /
266     > serving / fallback* only ? never as a *training* data source.** Harvesting their
outputs
267     > to train our owned model would breach their "no competing model" terms (a
terms/copyright
268     > violation), so it is prohibited in both engineering and legal process. **Training
teachers
269     > are open, Apache-2.0 models only** (Qwen-VL / InternVL / Gemma 4). Detail in the
270     > distillation constraint below.
271
272     ### Core principle: AI is a pluggable provider with a paid fallback
273     The codebase already abstracts the cloud AI behind `provider_registry` /
274     `AIVideoProvider` ( `backend/providers/` ). Make that a **first-class platform
principle**:
275     - Every generative capability routes through the **provider registry**.

```

```

276 - The **owned model** plugs in as a new provider (`owned_vlm`); **Gemini (paid, via
277 Vertex AI)** stays registered as the **fallback** (failover on error/low-confidence,
278 or as the quality ceiling/teacher reference).
279 - This lets us **ship today on the known paid stack** and **swap in the owned model when
280 its quality clears a measured bar** ? with zero architectural churn and the paid path
281 always one config flip away. (Mirrors the COMMERCIALIZATION C4 A/B/C strategy: the
282 owned model is the long-run "(A) eliminate the per-call dependency"; paid is "(C)".)
283
284 ### The model / training layer (new sub-layer, fed by the scaffolding)
285 `
286 Data curation      Training      Eval      Serving
287 (consented user vid (fine-tune an open, (extend the existing (self-host inference
288 + golden set + commercially-usable eval harness to QA as a provider /
289 rally/segment/event base VLM via quality + the ComputeBackend;
290 annotations ? LoRA/QLoRA) regression gate) paid fallback)
291 video-QA pairs)
292 `
293 This layer **reuses every other layer**: **Storage** holds the training corpus;
294 **Data** supplies labels (the `annotation` registry + `candidate_segments` +
295 golden set are already the labeling seam); **Compute/Orchestration** runs training and
296 inference jobs (even **BYO-compute** can serve inference on the tenant's GPU); the
297 **eval harness** already in `backend/eval/` extends to grade QA quality. **In other
298 words, the rock-solid scaffolding is exactly what makes the model flywheel possible** ?
299 which is why scaffolding comes first (see §6).
300
301 ### Base model & training (Apache-2.0 only ? licensing is decisive)
302 **Recommended near-term base: `Qwen2.5-VL-7B-Instruct`** ? Apache-2.0, native long-video
303 (dynamic-FPS sampling, temporal mRoPE), mature LoRA tooling. Upgrade path
304 **Qwen3-VL-4B/8B (Apache-2.0)**; **Gemma 4 multimodal (Apache-2.0, 2026)** is a clean
305 Google-stack alternative to evaluate.
306 - ?? **Lock to Apache-2.0-clean bases** so you can fine-tune, **keep weights private,
307 gate behind subscriptions, and never owe open-sourcing or branding**. (Note Qwen2.5-VL
308 is Apache **only at 7B/32B** ? 3B/72B use the custom Qwen license.)
309 - ? **Reject for the flagship:** **Gemma 3** ? custom terms with a **viral derivative
310 clause** (anything fine-tuned on it, *or even trained on its synthetic outputs*, stays
311 permanently bound to Google's *updatable* Prohibited-Use Policy); **Gemma 4's switch
312 to
313 Apache-2.0 removes this**. **Llama 3.2 Vision** ? custom license forces a "Llama-?"
314 model name + "Built with Llama" badge and a 700M-MAU ceiling. **gpt-oss`** ?
315 but **text-only**, so usable only as an optional text-reasoning layer, never the video
316 encoder.
317 - **Training method:** **QLoRA** (? r=64, ?=128) on the 7B ? fits one prosumer GPU and
318 avoids catastrophic forgetting of general video skills. Build the dataset by
319 converting
320 the existing rally/shot/segment annotations + golden set into templated **clip?QA
321 pairs**, expand synthetically with **open-model teachers only** (see constraint
322 below),
323 and hold out a **golden QA eval** that gates every fine-tune. A few hundred gold pairs
324 ?
325 **5k?20k** curated is a realistic band for this narrow domain.
326 - **Serving economics:** a 7B VLM ? 14 GB VRAM (FP16) / ~3.5 GB (INT4); a ?16 GB GPU is
327 a
328 comfortable serve target (vLLM for throughput). Self-hosting wins **only** once you
329 have
330 sustained video-QA volume **+** a defensible consented dataset **+** real per-call API
331 cost pain ? below that, the paid API is cheaper than a standing GPU and the second
332 stack
333 is overhead. Hence: **paid-primary now ? owned-primary / paid-fallback later.**
334
335 Base-model licensing + training-data policy are tracked as **COMMERCIALIZATION C14**.
336
337 ### Training backend ? managed service vs self-managed (the fine-tune "forge")
338 *Where* you run the LoRA/RL fine-tune. Weigh on **video support, weight
339 export/ownership,

```

```

332     smallest servable model, RL support, no-train DPA, and infra burden**.
333
334     **Decisive finding (researched 2026-06):** managed fine-tuning services are
335     **image-based** (JSONL + base64 images) ? for video you pre-sample frames and feed them
336     as image sequences. **Only the self-managed `ms-swift` explicitly advertises *native*
337     video (+audio) training.** So for true video-QA, a self-managed framework is the
338     strongest technical fit ? and it also lets you fine-tune the **small 7B** (cheap to
339     serve) and keep full ownership. Managed services trade some of that for zero infra.
340
341     | Option | Type | Video / VLM support | Smallest VL | RL | Export weights | Best for |
342     |---|---|---|---|---|---|
343     | **ms-swift** (ModelScope) | Self-managed (rented/own GPU) | **Native video+audio**;  
Qwen3-VL/Omni, 300+ MLLMs | any incl **7B** | ? DPO/GRPO | ? full ownership | *** video-QA;  
slots into BYO-compute ($3.2)** |
344     | **LLaMA-Factory** | Self-managed | Qwen3-VL + 100+ VLMs; video datasets | any incl 7B  
| ? | ? | popular, well-documented |
345     | **Unsloth** | Self-managed (1-GPU optimized) | Qwen3-VL LoRA (vision+lang; frames) |  
any incl 7B | ? GRPO | ? | cheapest first runs |
346     | **Tinker** (Thinking Machines) | Managed API (low-level) | Qwen3-VL **30B-A3B+ only**;  
image input (video **unverified**) | 30B-A3B MoE | ? PPO/GRPO, custom loss | ? adapters + ckpt;  
no-train ToS | zero-infra RL/research at MoE scale |
347     | **Together AI** | Managed (tune + serve) | Qwen3-VL / Gemma-3 / Llama-4; image data |  
verify | partial | ? adapters downloadable | managed Qwen3-VL + serving |
348     | **Fireworks AI** | Managed (tune + serve) | Full Qwen2.5-VL (3B?72B SFT; LoRA@32B +  
LoRA-upload) | **7B** | partial | ?? verify export (lock-in risk) | managed *small*-VL + serving  
|
349     | **Predibase** (Rubrik) | Managed *** VPC self-host** | VLM instruct-tune; **LoRAX**  
multi-adapter | verify | ? (VPC) | ? **RFT/GRPO** flagship | RL "creativity" + multi-adapter +  
privacy |
350     | ~~OpenAI GPT-4o / Gemini tuning~~ | Closed | ? | ? | ? | ? no export | **non-fit ?  
violates "own the model"*** |
351
352     *(Enterprise clouds ? Databricks Mosaic / SageMaker / Vertex on open models ? give full  
353     control but are only worth it if you're already on that stack.)*
354
355     **Recommendation:**
356     - **Early iteration / true video-QA ?** self-managed **`ms-swift`** (or LLaMA-Factory /  
357     Unsloth) on a **rented GPU** (RunPod / Lambda / Modal / SkyPilot-to-your-cloud):
native
358     video, any size incl. **7B** (cheap to serve), full ownership, and it drops into the
359     **ComputeBackend / BYO-compute** seam ($3.2). **Likely a better fit than Tinker** for
360     the video requirement *and* serving economics.
361     - **Managed, no GPU ops ?** **Together AI** (Qwen3-VL + downloadable adapters + serving)  
362     or **Fireworks** (full Qwen2.5-VL incl. 7B + serving) ? confirm frame/video support  
and
363     adapter export.
364     - **If RL "coaching creativity" + per-sport multi-adapter is central ?** **Predibase**  
365     (RFT + LoRAX, VPC self-host for the biometric/privacy posture) or **Tinker** (lowest-  
366     level RL primitives).
367     - **Tinker** remains a strong zero-infra RL-research option with clean weight export ?
just
368     not the cheapest path to a *small video* model (its VL floor is 30B-A3B).
369     - **Portability is the hedge:** you own the **data + recipe + exported weights**, so the  
370     backend is swappable ? same ethos as the provider/registry pattern. Don't get locked  
in.
371
372     **Verify before committing (any backend):** (1) **native video / frame-sequence**  
373     ingestion + max frames per example; (2) **weight export** + a **commercial DPA**  
covering
374     consented, minors-excluded video (biometric, $7); (3) the **inference plan** for the  
size
375     you train (a self-managed **7B** is far cheaper to serve than a 30B-A3B MoE).
376
377     ### Training harness ? make it agent-runnable (nanochat-style operability)
378     The goal: **an agent (or CI) can run a fine-tune on a fresh batch of golden-labeled

```

```

video
379     and get a trustworthy ship / no-ship verdict, unattended.** Karpathy's **nanochat** is
the
380     reference for that *operability* ? one MIT repo, one `speedrun.sh` that runs
381     tokenizer?train?eval?serve, a single complexity dial, and an automatic **report-card**
382     score. It is **not** a usable engine here (text-only, from-scratch GPT-2-class), but its
383     five properties are exactly what make training agent-runnable, and we copy them:
384     **(1)** one command, end-to-end; **(2)** one complexity dial (sane auto-derived
defaults);
385     **(3)** an automatic **report card** ? a deterministic pass/fail vs a target (the crux);
386     **(4)** reproducible + deterministic + cheap; **(5)** minimal/readable.
387
388     **Engine (don't reinvent):** **`ms-swift`** already delivers this for VLMs ? CLI-driven,
389     **native video**, Qwen3-VL support, an eval backend (EvalScope), and reproducibility via
390     `args.json` + `full_determinism`. (LLaMA-Factory is the close second; HF **`nanoVLM`** ?
391     ~750 lines ? is the minimal reference to *learn* VLM training, not the production path.)
392
393     **The harness = thin glue over components we already have** ? one declarative,
394     agent-invokable job (call it `train_speedrun`):
395     1. **Build dataset** from golden labels ? clip?QA pairs (reuse the `annotation` registry
+
396     `candidate_segments` + golden set ? the labeling seam in `backend/`).
397     2. **Fine-tune** via ms-swift on a **ComputeBackend** GPU (§3.2), run as an
orchestration
398     job (§5 / DEFERRED #16): one config, idempotent, logged.
399     3. **Auto-eval + report card** ? extend the existing `backend/eval/` regression harness
to
400     **QA quality**; emit a `report.md` pass/fail vs the **held-out golden QA eval**.
401     4. **Register** the result as the `owned_vlm` provider (Gemini paid fallback stays)
**only
402     if it beats the bar**.
403
404     So: **golden labels ? ms-swift train (on a compute backend) ? eval gate ? register
405     provider**, behind one command an agent or CI runs. nanochat is the inspiration for the
406     wrapper; ms-swift is the engine; the **golden QA eval is the report card**.
407
408     > **Prerequisite (the crux):** the harness is only as trustworthy as the **golden QA
eval
409     > set**. nanochat's speedrun is meaningful *because* its eval (CORE score) is automatic
and
410     > trustworthy; ours is the golden QA eval ? **build that first**, or an agent-run loop
411     > optimizes blind. (Same "moat = dataset + eval", scaffolding-first principle.)
412
413     ### ?? Teacher / distillation constraint (the single biggest legal landmine)
414     You **cannot** train the owned model on outputs from the paid frontier APIs ? a general
415     video-QA assistant is squarely a "competing model": **Gemini** ("may not use the
416     Services to develop models that compete"), **OpenAI** ("may not use Output to develop
417     models that compete"), **Anthropic** (bars using Outputs to train competing models). Two
418     safe lanes: **(1)** generate synthetic QA with **open-model teachers only** (Qwen-VL /
419     InternVL / Gemma 4 ? Apache-clean); **(2)** keep the paid API strictly as a **runtime
420     fallback** ? serving answers is fine, *harvesting its outputs to train the model that
421     replaces it is not*. Write a hard **"no training on competitor-API outputs"** rule into
422     eng + legal process. (This makes the ROADMAP's "Gemini as offline teacher only" precise:
423     the teacher is *open models*, not the paid APIs.)
424
425     ### Data flywheel, personalization & consent tiers
426     Two complementary loops, each with its **own** opt-in (training consent is separate from
427     processing consent ? §7):
428
429     - **Global model (shared ? raises the floor).** Train the base owned model on an
430     **explicitly-contributed, licensed** corpus ? *not* on whatever sits in a tenant's
bucket
431     (preserves "your video, your storage"). Flywheel: *more consented, labeled matches ? a
432     better shared model ? better highlights/answers ? more reason to contribute.* Higher
433     consent bar (pooled data, biometric/minor care ? §7); **minors excluded by default**.

```

```

434
435     - **Per-user adapters (personalized ? raises the ceiling) ? owner hunch, under
exploration.**
436     A small **per-user (or per-tenant) LoRA adapter, fine-tuned *only on that user's own
437 consented videos** on top of the shared base, served via **multi-LoRA** (LoRAX /
438 Together / Predibase). Why it's compelling:
439     - **Precision:** adapts to *that user's* court, camera angle, lighting, and players'
440 styles ? materially better detection/highlights for them.
441     - **Cleanest consent story:** "improve **your** detection using **your** data" is
far
442     lower-friction and lower-risk than contributing to a global model ? the data
improves
443     **only the user's own adapter**, with **no cross-user mixing**, sidestepping much of
the
444     pooled-corpus biometric/minor entanglement. Reinforces the differentiator: *your
video,
445     your storage, your compute ? and now **your model**. With BYO-compute the adapter
can be
446     trained/served on the user's own GPU and **never leave their boundary**; **deletion
=
447     drop the adapter** (clean erasure).
448     - **Open questions (still a hunch):** cold-start (need enough of the user's labeled
data
449     before a personal adapter beats the base ? fall back to the shared model meanwhile);
a
450     **per-user held-out eval** to prove the adapter helps (don't ship a regression);
base +
451     optional global-adapter + per-user-adapter stacking; per-user training cost at
scale.
452     Worth a spike once the harness + golden eval exist.
453
454     **Two consent tiers, then:** *(a)** "improve **my own** detection with **my** data"
455 (per-user, low friction) and *(b)** "contribute to the **shared** model" (global,
higher
456 bar). **Default for both is off.**
457
458     ## 6. Phased roadmap
459
460     | Phase | Deliverable | Build / reuse |
461     |---|---|---|
462     | **P0** | **Async job model, single-tenant** (DEFERRED #16) | `jobs` table via
`user_version` ladder; worker; 202+status endpoints |
463     | **P1** | **Identity + multi-tenant data model + upload + highlight persistence +
history** | auth (build vs Auth0/Clerk/Supabase ? decide); Postgres + `tenant_id`/RLS;
`highlights` table; repository layer; history API + UI |
464     | **P2** | **`StorageBackend` registry + BYO-storage** | new ABC+registry;
`local`/`gdrive`/`s3`/`gcs`; replace `video_path` with `StorageRef`; encrypted per-tenant creds
|
465     | **P3** | **`ComputeBackend` registry + BYO-compute** | `local`/`hosted`/`byo_worker`;
pull-based worker agent; control/data-plane split |
466     | **P4** | **Rich rally/shot semantic metadata & tagging** *(the next value stage after
solid rally detection + highlights)* | shot type (smash / backhand drive / net), rally
**outcome** (winner / forced / **unforced error**), context (e.g. **open court but error**) ?
query-API filters + `HighlightMetadata` + a **human correction/review loop** feeding the golden
set via the `annotation` registry + `candidate_segments` |
467     | **P5** | **Owned generative video-QA model** ($5A) | global flywheel **+ per-user
adapters**; P4 metadata is training signal ? fine-tune an open VLM (Qwen-VL) via LoRA ? serve as
`owned_vlm` **provider with Gemini paid fallback**; extend the eval harness to QA quality |
468     | **P6** *(visionary)* | **Gait / biomechanics under a supervising coach** |
deliberately **back-burner**; technique feedback; highest Art. 9 bar ? gate behind explicit
biometric + parental consent ($7) |
469
470     P0?P1 deliver the headline ask (per-user upload + stored highlights + history). P2?P3
471     deliver the "plug-and-play storage/compute" ask. **P4 (rich semantic metadata) is the
472     immediate next value stage once rally detection + highlights are solid** ? e.g.

```

```

*rallies
473     that ended in a smash or backhand drive,"* *rallies where the court was open but the
474     player made an unforced error."* *P5** owns the model; *P6 (gait) is the visionary
475     horizon, intentionally deferred.**
476     **Sequencing principle (owner directive):** make the scaffolding (P0?P4) **rock-solid
477     first**, keep the **known paid stack as an always-ready fallback**, and only then shift
478     focus to **owned-model quality** (P5). The provider registry keeps the paid path one
479     config flip away.
480
481     ## 7. Privacy & legal (gait raises the bar ? design for it now)
482
483     - **The load-bearing nuance:** biometric data is special-category **only when used to
484     *uniquely identify* a person** (GDPR Art. 9). **Highlight/shot detection on players a
485     coach already knows is general analytics ? generally *not* Art. 9.** But
486     **gait/biomechanics that builds a per-athlete identifying template tips into Art. 9**
487     and needs **explicit, granular consent** referencing biometric data. Engineer gait
488     with that boundary (consent gate, purpose limitation, no cross-account
identification);
489     decide whether it runs **locally/on-tenant only** (strong posture) vs our cloud. (EDPB
490     Guidelines 3/2019 on video devices apply.)
491     - **Minors (e.g. academies).** Parental/guardian consent and safeguarding are first-
order,
492     not an afterthought. Model consent explicitly per relationship (user?subject,
coach?athlete
493     where the overlay applies, parent?minor).
494     - **BYO-storage is the central privacy/liability lever:** if raw video stays in the
495     tenant's own bucket/Drive and we process via signed URL or their compute, we minimize
496     what we hold ? shrinking breach surface, residency, and erasure obligations.
497     - **Paid-LLM no-train (C4):** use **Gemini via Vertex AI** (contractual no-training),
498     **not the consumer/free tier** ; set short/zero retention, document it in the DPA, and
499     surface "your footage is never used to train AI" as a trust feature for academies. For
500     the gait phase, prefer keeping biometric inference off third-party APIs entirely.
501     - **Training consent is a distinct, higher bar (for the owned model, §5A).** Training on
502     user video needs a **separate, explicit, opt-in training licence** ? not bundled with
503     processing consent ? and it **compounds** the biometric (Art. 9) and minor-consent
504     issues above. Train **only on explicitly contributed/licensed video** ; never train on
505     what merely passes through a tenant's bucket. **Exclude minors' video from the
training
506     pool by default** , keep the training corpus a **separate, firewalled, opt-in sub-
pool** ,
507     and log **per-clip consent/provenance** for revocation + audit. Incentives must not
make
508     consent non-free (the product stays fully usable without contributing). And the
509     **teacher-distillation rule** (§5A): never train on paid-LLM outputs. **Two consent
tiers
510     (§5A):** *(a)* "improve **my own** detection with **my** data" ? a per-user adapter
trained
511     only on the user's own video (lowest friction; no cross-user mixing; deletion = drop
the
512     adapter); *(b)* "contribute to the **shared** model" ? pooled corpus, higher bar.
Default
513     for both is **off** .
514     - **Right to erasure / retention:** per-tenant, per-athlete delete must cascade across
DB +
515     storage artifacts (the `ON DELETE CASCADE` discipline already in `database.py` extends
here).
516
517     ## 8. Key decisions for the owner (the forks)
518
519     1. **Hosted-first vs BYO-first.** Recommended: **hosted control-plane first** (P0?P1)
with
520     **BYO-storage soon after** (P2); **BYO-compute later** (P3) as a power-
user/enterprise
521     option ? it's the highest-complexity, highest-security item.
522     2. **Database now or later.** Move to **Postgres at P1** for the hosted path, or stay

```

523 SQLite-per-tenant if self-hosted-single-academy is the first GTM. (Abstract DB access
524 regardless.)

525 3. **Auth: build vs buy** (Auth0 / Clerk / Supabase Auth / Cognito).

526 4. **Orchestration tech** (see research recommendation) ? bias to the simplest that
gives
527 retries + status.

528 5. **Gait processing location** ? local/on-tenant (privacy-max) vs cloud (capability-
max).

529 6. **How far to push BYO-compute** given its security/ops complexity.

530 7. **Owned-model base + licensing (\$5A).** Which commercial-OK video VLM to build on
531 (Qwen-VL Apache-2.0 vs Gemma 3 custom license vs ?) and confirm a gated commercial
532 derivative is permitted ? see COMMERCIALIZATION C14.

533 8. **Training-data sourcing & consent.** Opt-in contribution terms, incentives, and the
534 teacher-distillation constraint (whether paid-LLM outputs may seed the corpus).

535 9. **When to start the owned model** ? strictly after the scaffolding + flywheel are
536 solid (the explicit directive), with the paid provider as the standing fallback.

537

538 **Risks:** scope explosion (sequence strictly P0?P5); multi-tenant data-leak bugs (RLS +
539 tests); BYO-compute security; **biometric/minor liability** (get this right before any
540 gait feature ships); **training on user video without proper opt-in/licence**, and
541 **base-model / teacher-distillation licence violations**; cost of hosted GPU (training
542 and inference); **premature model-building** before data/eval are ready (mitigated by
543 keeping the paid fallback and gating P5 behind the flywheel).

544

545 **## 9. Sources**

546

547 External best-practices research (2026-06-07) backing the recommendations above. Each
548 underlying claim in the research was tagged fact-vs-synthesis; figures are as-of and
549 should be re-validated before committing budget.

550

551 **Storage abstraction:** [fsspec](https://fsspec.com/) ·
[cloudpathlib](https://github.com/drivendataorg/cloudpathlib) ·
[smart_open](https://github.com/piskvorky/smart_open) · [Apache
Libcloud](https://libcloud.apache.org/) · [gdrive-fsspec](https://github.com/fsspec/gdrive-
fsspec) · [dropboxdrivefs](https://github.com/fsspec/dropboxdrivefs) · [AWS S3 presigned
URLs](https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html) · [token-
based S3 with STS](https://oneuptime.com/blog/post/2026-02-12-token-based-access-s3-sts/view) ·
[signed URLs across clouds](https://medium.com/@shaikhsarwan49/signed-urls-in-aws-gcp-and-azure-
a-beginner-friendly-guide-e2e549425865)

552

553 **Multi-tenancy:** [Bytebase: multi-tenant DB
patterns](https://www.bytebase.com/blog/multi-tenant-database-architecture-patterns-explained/)
· [Hunchbite: RLS vs schema-per-tenant](https://hunchbite.com/guides/multi-tenant-saas-
architecture) · [PlanetScale: tenancy in Postgres](https://planetscale.com/blog/approaches-to-
tenancy-in-postgres) · [DEV: DB-per-tenant vs shared schema](https://dev.to/young_gao/multi-
tenant-architecture-database-per-tenant-vs-shared-schema-1n2e)

554

555 **BYO-compute / orchestration:** [Pinecone BYOC](https://www.pinecone.io/blog/byoc/) ·
[Nuon: control/data-plane](https://nuon.co/blog/byoc-control-plane-data-plane-architectures) ·
[GitHub self-hosted runners (ARC)](https://docs.github.com/en/actions/hosting-your-own-
runners/managing-self-hosted-runners-with-actions-runner-controller/about-actions-runner-
controller) · [SkyPilot](https://github.com/skypilot-org/skypilot) · [SkyPilot AI control
plane](https://blog.skypilot.co/ai-job-orchestration-pt2-ai-control-plane/) · [Modal (AWS
Marketplace)](https://aws.amazon.com/marketplace/pp/prodview-j727623xqhh2k) · [Northflank:
Anyscale alternatives](https://northflank.com/blog/anyscale-alternatives-for-ai-ml-model-
deployment) · [DevPro: Python task queues
2025](https://devproportal.com/languages/python/python-background-tasks-celery-rq-dramatiq-
comparison-2025/) · [Judoscale: choosing a task queue](https://judoscale.com/blog/choose-python-
task-queue) · [Hatchet vs Temporal](https://hatchet.run/versus/hatchet-vs-temporal) ·
[Hatchet](https://github.com/hatchet-dev/hatchet)

556

557 **Privacy / legal:** [GDPR Art. 9](https://gdpr-info.eu/art-9-gdpr/) · [GDPR Local:
biometric compliance](https://gdprlocal.com/biometric-data-gdpr-compliance-made-simple/) ·
[Legiscope: Art. 9](https://www.legiscope.com/blog/gdpr-article-9-special-categories.html) ·
[EDPB Guidelines 3/2019 (video)](https://www.edpb.europa.eu/sites/default/files/files/file1/edpb

_guidelines_201903_video_devices_en_0.pdf) · [Fox Rothschild: EDPB video guidance](https://www.foxrothschild.com/publications/video-surveillance-under-gdpr-edpb-issues-final-guidance) · [ICO: children & UK GDPR](https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/childrens-information/children-and-the-uk-gdpr-old/what-are-the-rules-about-an-iss-and-consent/) · [EDPB Guidelines 05/2020 (consent)](https://www.edpb.europa.eu/sites/default/files/files/file1/edpb_guidelines_202005_consent_en.pdf) · [AI data-retention compared](https://ax-sentinel.com/blog/ai-data-retention-policies-compared) · [Xenoss: enterprise LLM platforms](https://xenoss.io/blog/openai-vs-anthropic-vs-google-gemini-enterprise-llm-platform-guide)

558

559 **Comparable products:** [Veo](https://www.veo.com/) · [Veo Help Center](https://support.veo.com/hc/en-us/articles/4459159526929-Overview-of-cam-veo-co-Record-manage-and-upload-footage-with-your-Veo-Cam-1) · [Latterly: Hudl competitors](https://www.latterly.org/hudl-competitors/) · [Apple: Behind the Design ? SwingVision](https://developer.apple.com/news/?id=0pg4dthn) · [SwingVision FAQ](https://swing.vision/faq) · [Sportsvisio: coach video-analysis comparison](https://www.sportsvisio.com/stories/video-analysis-software-for-coaches-complete-comparison)

560

561 **Training backends & harness (\$5A):** [karpathy/nanochat (harness reference)](https://github.com/karpathy/nanochat) · [HF nanoVLM (minimal VLM)](https://github.com/huggingface/nanoVLM) · [nanoVLM blog](https://huggingface.co/blog/nanovlm) · [Tinker (GA + vision)](https://thinkingmachines.ai/blog/tinker-general-availability/) · [Tinker model lineup](https://tinker-docs.thinkingmachines.ai/model-lineup) · [Tinker ToS (weights/no-train)](https://thinkingmachines.ai/legal/tos/) · [Fireworks VLM fine-tuning](https://fireworks.ai/blog/vlm-tuning) · [Together AI fine-tuning](https://www.together.ai/fine-tuning) · [Predibase: fine-tune & serve VLMs](https://predibase.com/blog/how-to-fine-tune-and-serve-vlms-in-predibase) · [Predibase RFT/GRPO](https://docs.predibase.com/user-guide/fine-tuning/grpo) · [ms-swift (native video/audio)](https://github.com/modelscope/ms-swift) · [LLaMA-Factory (VLMs)](https://github.com/hiyouga/LlamaFactory) · [Unsloth (Qwen3-VL)](https://qwen.readthedocs.io/en/latest/training/unsloth.html) · [Fine-tuning tools 2026 (LoRA/QLoRA)](https://www.index.dev/blog/top-ai-fine-tuning-tools-lora-vs-qlora-vs-full)

562

563 **Owned model & licensing (\$5A):** [Qwen2.5-VL](https://qwenlm.github.io/blog/qwen2.5-vl/) · [Qwen2.5 license split](https://qwenlm.github.io/blog/qwen2.5/) · [Qwen3-VL](https://github.com/QwenLM/Qwen3-VL) · [Gemma 3 (multimodal)](https://huggingface.co/blog/gemma3) · [Gemma Terms of Use](https://ai.google.dev/gemma/terms) · [Gemma license-risk analysis](https://wcr.legal/google-gemma-license-risks/) · [Gemma 4 / Apache-2.0](https://www.mindstudio.ai/blog/what-is-gemma-4-apache-2-license-commercial-ai-deployment) · [Llama 3.2 license (700M MAU + naming)](https://www.llama.com/llama3_2/license/) · [gpt-oss (text-only, Apache-2.0)](https://openai.com/index/introducing-gpt-oss/) · [Gemini API terms (compete clause)](https://ai.google.dev/gemini-api/terms) · [OpenAI terms](https://openai.com/policies/row-terms-of-use/) · [Anthropic: outputs & training](https://support.claude.com/en/articles/12326764-can-i-use-my-outputs-to-train-an-ai-model) · [QLoRA practical](https://apxml.com/courses/fine-tuning-adapting-large-language-models/chapter-4-parameter-efficient-fine-tuning/practice-qlora-fine-tuning) · [VLM LoRA cookbook](https://huggingface.co/learn/cookbook/en/fine_tuning_vlm_trl) · [GDPR Art. 9](https://gdpr-info.eu/art-9-gdpr/) · [athletes & biometric data](https://www.architectureandgovernance.com/applications-technology/biometric-data-and-athletes-privacy-law-and-compliance-implications/) · [federated learning + DP](https://research.google/blog/federated-learning-with-formal-differential-privacy-guarantees/) · [self-hosted LLM VRAM/cost](https://www.gpu-mart.com/guides/self-hosted-llm) · [Gemini API pricing](https://ai.google.dev/gemini-api/docs/pricing)

564

565 ## See also

566 - [``COMMERCIALIZATION.md``](COMMERCIALIZATION.md) ? C12 (SaaS foundations), C4 (paid-LLM), economics.

567 - [``DEFERRED\_HARDENING\_PLAN.md``](DEFERRED_HARDENING_PLAN.md) ? #16 async job model (the P0 infra slice), #7 packaging.

568 - [``PMF\_MARKET\_RESEARCH.md``](PMF_MARKET_RESEARCH.md) ? academy/coach beachhead (who this serves).

569 - [``CODE\_MAP.md``](CODE_MAP.md) ? current architecture + the registry extension points.

570 - [`ROADMAP.md`](ROADMAP.md) ? offline self-improving rally detection (the pipeline's
own trajectory).
571

10. assistant (2026-06-09T11:26:04.475Z)

I have a thorough picture of the codebase posture. The existing strategy already leans Python-everywhere (FastAPI, ms-swift, fsspec, Arq/Hatchet ? all Python). My job is to give a language recommendation for the reframed vision (quality-first training harness, multisport, conversational-QA with per-video bookkeeping). Let me verify the current-fact questions via web research: (1) is ms-swift Python-locked / what's the language story of the training+eval layer, (2) on-device export runtimes and whether they dictate dev language, (3) Python async-job/hot-path reality for the platform.

11. user (2026-06-09T11:26:16.145Z)

Web search results for query: "ms-swift fine-tuning framework Python CLI command line interface 2026"

Links: [{"title": "Pre-training and Fine-tuning ? swift 4.3.0.dev0 documentation", "url": "https://swift.readthedocs.io/en/latest/Instruction/Pre-training-and-Fine-tuning.html"}, {"title": "ms-swift · PyPI", "url": "https://pypi.org/project/ms-swift/"}, {"title": "GitHub - modelscope/ms-swift: Use PEFT or Full-parameter to CPT/SFT/DPO/GRPO 600+ LLMs (Qwen3.6, DeepSeek-V4, GLM-5.1, InternLM3, Llama4, ...) and 300+ MLLMs (Qwen3-VL, Qwen3-Omni, InternVL3.5, Ovis2.5, GLM4.5v, Gemma4, Llava, Phi4, ...) (AAAI 2025).", "url": "https://github.com/modelscope/ms-swift"}, {"title": "MS-SWIFT | AI Native Landscape", "url": "https://jimmysong.io/ai/ms-swift/"}, {"title": "Quick Start ? swift 4.4.0.dev0 documentation", "url": "https://swift.readthedocs.io/en/latest/GetStarted/Quick-start.html"}, {"title": "MS-SWIFT - Qwen", "url": "https://qwen.readthedocs.io/en/latest/training/ms_swift.html"}, {"title": "GitHub - maochaojie/swift: SWIFT (Scalable lightWeight Infrastructure for Fine-Tuning) is an extensible framwork designed to facilitate lightweight model fine-tuning.", "url": "https://github.com/maochaojie/swift"}, {"title": "Pre-training and Fine-tuning ? swift 3.12.6 documentation", "url": "https://swift.readthedocs.io/en/v3.12/Instruction/Pre-training-and-Fine-tuning.html"}, {"title": "modelscope/ms-swift | DeepWiki", "url": "https://deepwiki.com/modelscope/ms-swift"}, {"title": "Pre-training and Fine-tuning ? swift 3.6.4 documentation", "url": "https://swift.readthedocs.io/en/v3.6/Instruction/Pre-training-and-Fine-tuning.html"}]

Based on the search results, here's information about the ms-swift fine-tuning framework and its Python CLI command line interface:

Overview

ms-swift is a large model and multimodal large model fine-tuning and deployment framework provided by the ModelScope community that now supports training (pre-training, fine-tuning, human alignment), inference, evaluation, quantization, and deployment for 600+ text-only large models and 400+ multimodal large models.

CLI Interface Design

ms-swift employs a hierarchical design philosophy, allowing users to perform fine-tuning through the command line interface, Web-UI interface, or directly using Python.

Recent Updates (2026)

As of 2026.03.03, ms-swift v4.0 major version is officially released.

CLI Command Examples

The framework provides various CLI commands for different tasks. Example commands include `swift sft` for supervised fine-tuning, `swift rlhf` for reinforcement learning, `swift infer` for inference, `swift app` for creating interactive applications, `swift deploy` for deployment,

`swift sample` for sampling, and `swift eval` for evaluation.

A typical fine-tuning command example shows the use of parameters like `--model`, `--tuner\_type`, `--dataset`, `--output\_dir`, and various training hyperparameters.

Training Capabilities

The framework supports lightweight fine-tuning methods such as LoRA, QLoRA, DoRA, LoRA+, LLaMAPro, LongLoRA, LoRA-GA, ReFT, RS-LoRA, Adapter, LISA, etc.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

12. user (2026-06-09T11:26:17.118Z)

Web search results for query: "ONNX Runtime cross-language C++ C# Rust Python bindings exported model inference 2026"

Links: [{"title":"ONNX Runtime | Home","url":"https://onnxruntime.ai/"},"{"title":"onnxruntime - Rust","url":"https://docs.rs/onnxruntime/latest/onnxruntime/"},"{"title":"Rust, ORT, ONNX: Real-Time YOLO on a Live Webcam- Part 1 | by Alfred Weirich | Medium","url":"https://medium.com/@alfred.weirich/rust-ort-onnx-real-time-yolo-on-a-live-webcam-part-1-b6edfb50bf9b"},"{"title":"ONNX Runtime | onnxruntime","url":"https://onnxruntime.ai/docs/"},"{"title":"ONNX Runtime Execution Providers","url":"https://onnxruntime.ai/docs/execution-providers/"},"{"title":"GitHub - microsoft/onnxruntime: ONNX Runtime: cross-platform, high performance ML inferencing and training accelerator · GitHub","url":"https://github.com/microsoft/onnxruntime"},"{"title":"ONNX Runtime: Enabling Cross-Platform AI Model Inference | by Everton Gomedede, PhD | Medium","url":"https://medium.com/@evertongomedede/onnx-runtime-enabling-cross-platform-ai-model-inference-80f136ecbb2d"},"{"title":"PyTorch Inference | onnxruntime","url":"https://onnxruntime.ai/docs/tutorials/accelerate-pytorch/pytorch.html"},"{"title":"ONNX Runtime | Inference","url":"https://onnxruntime.ai/inference"},"{"title":"microsoft/onnxruntime | DeepWiki","url":"https://deepwiki.com/microsoft/onnxruntime"}]

Based on the search results, here's what I found about ONNX Runtime's cross-language support for C++, C#, Rust, and Python bindings for model inference:

Language Bindings Support

ONNX Runtime has support for many languages, specifically APIs in C++, Python, C#, Java, Javascript, Julia, and Ruby. Models can be run in C++, Rust, Java, C#, or Go via ONNX Runtime.

Rust Bindings

For Rust specifically, ONNX Runtime can be accessed through the `ort` crate, which provides a safe and idiomatic interface to the ONNX Runtime C API. The unsafe C API is wrapped using `bindgen` as `onnxruntime-sys`, and the unsafe bindings are wrapped to expose a safe API.

Model Export and Cross-Language Inference

A YOLOv8 object detection model can be trained in Python, exported as an `.onnx` file, and then executed efficiently in Rust or C++ without any Python dependency. Models can be trained from any framework that supports export/conversion to ONNX format.

Consistent API Across Languages

The same ONNX Runtime API is used across all execution providers, providing a consistent interface for applications to run with different hardware acceleration platforms, and the APIs to set EP options are available across Python, C/C++/C#, Java and `node.js`.

The search results show that ONNX Runtime continues to be actively maintained as a robust solution for cross-language ML model inference in 2026.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

13. assistant (2026-06-09T11:26:22.467Z)

Both confirm key facts: ms-swift is Python/CLI-driven (the training/eval layer is Python-locked in practice but invoked via CLI, so it's a subprocess regardless of caller language), and ONNX Runtime decouples export from runtime language (the exported model is language-agnostic ? a Rust/C++/Swift host can serve it without Python). Let me confirm two more current facts: the on-device runtime story (llama.cpp / Core ML / ExecuTorch for VLMs, since the eventual compression target is a VLM) and the Python async/concurrency reality for FastAPI hot paths (free-threading / GIL status in 2026).

14. user (2026-06-09T11:26:31.465Z)

Web search results for query: "on-device VLM inference llama.cpp ExecuTorch MLX Core ML quantized vision language model mobile 2026"

Links: [{"title":"On-Device LLMs: State of the Union, 2026","url":"https://v-chandra.github.io/on-device-llms/"},"{"title":"On-Device LLMs in 2026: What Changed, What Matters, What's Next - Edge AI and Vision Alliance","url":"https://www.edge-ai-vision.com/2026/01/on-device-llms-in-2026-what-changed-what-matters-whats-next/"},"{"title":"The Edge LLM Runtime Stack 2026: llama.cpp vs Ollama vs TensorRT Edge-LLM vs ExecuTorch vs vLLM vs MLX","url":"https://edgeaistack.ai/blog/edge-llm-runtime-stack-2026/"},"{"title":"2026 Mac Inference Framework Selection: vllm-mlx vs. Ollama vs. llama.cpp | MACGPU Blog","url":"https://macgpu.com/en/blog/2026-mac-inference-framework-vllm-mlx-ollama-llamacpp-benchmark.html"},"{"title":"llama.cpp vs MLX vs Ollama vs vLLM: Local AI Inference for Apple Silicon in 2026 ? Contra Collective","url":"https://contracollective.com/blog/llama-cpp-vs-mlx-ollama-vllm-apple-silicon-2026"},"{"title":"MobileVLM: A Fast, Strong and Open Vision Language Assistant for Mobile Devices","url":"https://arxiv.org/html/2312.16886v1"}]

Based on the web search results, here's what I found about on-device VLM inference with these tools and approaches in 2026:

Runtime Recommendations for Different Targets

ExecuTorch is recommended for mobile production, llama.cpp for desktop/prototyping, and MLX for the Apple ecosystem. ExecuTorch hit 1.0 GA in October 2025, with a 50KB base footprint and support for 12+ hardware backends.

Vision-Language Models

Vision-language models have shrunk dramatically, with SmolVLM-256M using under 1GB memory and outperforming models 300x its size. MiniCPM-V achieves frontier-level performance while running on phones.

MLX-VLM extends MLX to vision-language models.

Quantization and Optimization

Quantized models in GGUF format from HuggingFace (Llama 3.2 or Gemma 3) can be validated with llama.cpp before moving to ExecuTorch for production. Major labs have converged on efficient models like Llama 3.2 (1B/3B), Gemma 3 (down to 270M), and others targeting efficient on-device deployment.

Key Constraints and Solutions

The KV cache grows linearly with sequence length and can exceed model weight memory for long-

context inference, making KV cache management often more impactful than weight quantization.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

15. user (2026-06-09T11:26:33.511Z)

Web search results for query: "Python 3.13 3.14 free-threading no-GIL FastAPI async I/O bound performance 2026 production"

Links: [{"title": "Python GIL vs No-GIL: Real FastAPI benchmarks with free-threaded Python 3.13 | by Keval Dekivadiya | Apr, 2026 | Medium", "url": "https://medium.com/@kevaldekivadiya2415/python-gil-vs-no-gil-real-fastapi-benchmarks-with-free-threaded-python-3-13-b5751f8d57a2"}, {"title": "Python 3.13's Free-Threaded Mode: What No-GIL Actually Means for Your Code - Java Code Geeks", "url": "https://www.javacodegeeks.com/2026/06/python-3-13s-free-threaded-mode-what-no-gil-actually-means-for-your-code.html"}, {"title": "Python 3.14 Free-Threading in Django: Real Benchmarks for Async + Thread-Safe Views (No GIL Pain) | by Yogeshkrishnanseeniraj | Django Journal | Medium", "url": "https://medium.com/django-journal/python-3-14-free-threading-in-django-real-benchmarks-for-async-thread-safe-views-no-gil-pain-82edc73a1c20"}, {"title": "Python Without GIL: Real Performance Testing in Python 3.13 Free-Threading | by inprogrammer | Medium", "url": "https://medium.com/@inprogrammer/python-without-gil-real-performance-testing-in-python-3-13-free-threading-5bdb34f73125"}, {"title": "Python 3.14 Free-Threading and Experimental JIT: How Python Finally Breaks the GIL Barrier in 2026 - Sean Kim ? Arts and Tech", "url": "https://blog.imseankim.com/python-3-14-free-threading-jit-compiler-gil-removal-2026/"}, {"title": "Scaling asyncio on Free-Threaded Python | Labs", "url": "https://labs.quansight.org/blog/scaling-asyncio-on-free-threaded-python"}, {"title": "Breaking the Chains of the GIL in Python 3.14", "url": "https://dzone.com/articles/breaking-the-chains-of-the-gil-in-python"}, {"title": "Python 3.14: Free-Threading, GIL Removal, and Performance Boosts", "url": "https://www.webpronews.com/python-3-14-free-threading-gil-removal-and-performance-boosts/"}, {"title": "How Much FASTER Is Python 3.13 Without the GIL?", "url": "https://www.deciphr.ai/podcast/how-much-faster-is-python-313-without-the-gil"}, {"title": "Python's Liberation: The GIL is Finally Optional (And Why This Changes Everything) | by Aftab | Medium", "url": "https://medium.com/@aftab001x/pythons-liberation-the-gil-is-finally-optional-and-why-this-changes-everything-5579b43e969c"}]

Based on the web search results, here's what I found about Python 3.13/3.14 free-threading, no-GIL, FastAPI, and async I/O bound performance in 2026:

Key Findings

****CPU-Bound vs. I/O-Bound Performance:****

CPU-bound endpoints show roughly an 8x increase in performance from ~4 req/s to ~32 req/s with no code changes, just a different Python build, as free-threading enables multiple requests to execute in parallel across CPU cores. However, I/O results are nearly identical across both runtimes, as the GIL was never the bottleneck for blocking I/O operations. If your FastAPI app is mostly database queries and HTTP calls, you shouldn't expect a difference from free-threading.

****FastAPI Support and Ecosystem:****

FastAPI 0.136.0 officially supports free-threaded Python. Major libraries including NumPy, SciPy, and FastAPI now offer pre-compiled wheels for the free-threaded build, with a community tracker at py-free-threading.github.io for checking compatibility.

****Python 3.13 vs 3.14 Trade-offs:****

In Python 3.13, enabling free-threading caused single-threaded code to run approximately 40% slower than the standard build, but Python 3.14 addressed this by re-enabling the specialising adaptive interpreter, bringing that penalty down to just 5-10%.

****Asyncio Scaling:****

Python 3.14 implements changes using lock-free data structures and per-thread state for asyncio, allowing highly efficient task management and execution across multiple threads with linear performance scaling and no lock contention in multi-threaded scenarios.

****Production Readiness:****

For building a new service with CPU-intensive processing, benchmarking against the free-threaded build (ideally Python 3.14t given its improved single-thread overhead) is worthwhile, as potential gains are real and setup cost is low.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

16. assistant (2026-06-09T11:26:42.753Z)

I now have all the current facts I need. The picture is consistent and clear:

1. ****ML training/eval**** ? ms-swift is Python+CLI (subprocess-invokable; PyTorch ecosystem is Python-locked, but that's irrelevant to the orchestrator's language since you shell out to `swift sft`).
2. ****On-device**** ? ONNX Runtime / ExecuTorch / Core ML / llama.cpp all decouple the exported model from the runtime host language. The export is language-agnostic; on-device dictates the **deployment** runtime (Swift for iOS, C++ for ExecuTorch), not the **dev/training** language.
3. ****Platform hot paths**** ? Python 3.14 free-threading + async closes most of the historical GIL gap; video work is ffmpeg/GPU-subprocess-bound (already out-of-process), so there's no Python hot path to rewrite.

Let me note one important nuance about the on-device target: the strategy doc names Qwen3-VL-4B as the eventual base, and the search confirms `Gemma 3 270M` GGUF is mentioned for on-device ? but Gemma 3 is license-rejected per the guardrail. That's a data point, not a contradiction for my dimension. I have enough to synthesize. Let me record the final structured answer.

17. user (2026-06-09T11:28:02.129Z)

Structured output provided successfully

18. assistant (2026-06-09T11:28:15.116Z)

I've completed the analysis and returned my recommendation via StructuredOutput.

Bottom line: ****stay Python everywhere now****, and treat "another language" as a deferred, layer-local deployment detail rather than a rewrite or a hot-path split.

The three layers each independently point this way:

1. ****ML training+eval**** ? effectively Python-locked (PyTorch + ms-swift CLI + ActionFormer/ASFormer + your existing numpy LogReg Gen-0 seed), and that's free: training is offline/batch, not a latency hot path. ms-swift is invoked as a `swift sft` subprocess anyway, so the orchestrator's language is irrelevant to it.
2. ****Platform**** ? no real Python hot path to escape. Heavy bytes/GPU work already runs out-of-process (ffmpeg Windows-side, WASB in WSL2, cloud GPU per-job). The control plane is I/O-bound, where the GIL never bound you, and Python 3.14 free-threading + FastAPI 0.136 close the residual CPU objection. The Python-native stack already chosen in `PLATFORM\_ARCHITECTURE.md` (fsspec/smart_open, Arq/Hatchet, Postgres) covers it.
3. ****On-device (LATER)**** ? the only layer where a second language eventually appears, but it's structurally decoupled by the ****export boundary****: train in Python ? export to ONNX/Core ML/ExecuTorch/GGUF ? write a thin native runtime shell (Swift/Kotlin/C++/Rust-`ort`) only at mobile-ship time. On-device dictates a future **deployment** runtime, not your **dev** language.

The conversational-QA north star and per-video bookkeeping reinforce Python-everywhere ? durable relational state + a retrieval/tool layer is squarely FastAPI/Python, and splitting languages would fracture the one codebase holding QA state, the eval harness, and the provider registry

together.

Relevant files I grounded this in (all absolute):

- `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM\_ARCHITECTURE.md`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\OWNED\_MODEL\_IMPLEMENTATION\_PLAN.md`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\research\OWNED\_MODEL\_TRAINING\_STUDY.md`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE\_MAP.md`

One concrete action it implies (a model-architecture constraint, not a dev-language one): train with export-clean / ONNX-friendly ops so the eventual compression target stays reachable without re-architecting.